

Abstract

Last-mile delivery, the final leg of a product’s journey from a distribution hub to the customer’s doorstep, is consistently reported as the most expensive and least efficient segment of the logistics chain. For small and mid-sized delivery operations that cannot justify the recurring cost of commercial routing APIs, route planning is often done manually or through spreadsheets, leading to longer travel distances, missed delivery windows, and higher fuel consumption.

This report documents the design, implementation, and deployment of the Smart Route Optimization System, a full-stack web application built to address this problem for fleet operators managing a moderate number of vehicles and daily delivery stops. The system allows dispatchers to upload delivery locations through a CSV file or enter them manually, select an optimization strategy, and generate a delivery route that is ordered using either a nearest-neighbor heuristic or a priority-based scheduling strategy.

A central design decision in this project was to avoid dependency on paid mapping services such as the Google Maps Directions API. Instead, the system integrates with a self-hosted instance of the Open Source Routing Machine (OSRM) to obtain real road distances, travel durations, and road geometry for every generated route. When the OSRM service is temporarily unavailable, the system automatically falls back to a straight-line (Haversine) distance calculation so that route generation is never blocked, at the cost of reduced accuracy until the routing service is restored.

The backend is implemented using Node.js, Express, and TypeScript, with Prisma ORM managing a PostgreSQL database and Redis used for caching and background job coordination. The frontend is a React and TypeScript single-page application built with Vite and styled using Tailwind CSS, using Leaflet for map rendering and Chart.js for analytics visualization. Real-time vehicle location updates are propagated using Socket.IO.

The system supports role-based access control across three roles – Administrator, Dispatcher, and Driver – and includes modules for vehicle management (including weight and volume capacity tracking), driver management (with driver assignment performed by email lookup rather than raw database identifiers), route planning, live tracking, and an analytics dashboard summarizing fleet-wide distance, travel time, and delivery completion statistics.

The application was tested using unit tests for the core optimization algorithms, API-level testing using Postman, and load and performance testing using k6 across increasing levels of concurrent users. The system was deployed using Docker Compose, with the frontend hosted on Vercel and the backend hosted on Render, backed by a managed PostgreSQL instance.

This report presents the complete requirement analysis, system architecture, database design, algorithmic approach, implementation details, testing methodology and results, deployment process, and a discussion of the system's performance characteristics and limitations, followed by a set of proposed future enhancements.

Keywords: Route Optimization, Last-Mile Delivery, OSRM, Fleet Management, Nearest Neighbor Algorithm, Node.js, React, PostgreSQL, Docker.

Contents

List of Figures	6
List of Tables	8
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Motivation	2
1.4 Objectives	3
1.5 Scope	3
1.6 Expected Benefits	4
2 Requirement Analysis	6
2.1 Functional Requirements	6
2.2 Non-Functional Requirements	7
2.3 User Requirements	8
2.4 Business Requirements	9
2.5 Constraints	9
2.6 Assumptions	10
3 Technology Stack	11
3.1 Frontend Technologies	11
3.1.1 React with TypeScript	11
3.1.2 Vite	11
3.1.3 Tailwind CSS	12

3.1.4	React Router	12
3.1.5	Axios	12
3.1.6	React Query	13
3.1.7	React Hook Form	13
3.1.8	Leaflet and React Leaflet	13
3.1.9	Socket.IO Client	14
3.1.10	Chart.js	14
3.2	Backend Technologies	14
3.2.1	Node.js and Express	14
3.2.2	TypeScript	14
3.2.3	Prisma ORM	15
3.2.4	PostgreSQL	15
3.2.5	Redis	15
3.2.6	JWT and Bcrypt	16
3.2.7	Socket.IO (Server)	16
3.2.8	Multer	16
3.3	Routing Engine	16
3.3.1	OSRM (Open Source Routing Machine)	16
3.4	Deployment Technologies	17
3.4.1	Docker and Docker Compose	17
3.4.2	Render	17
3.4.3	Vercel	17
3.5	Summary	18
4	System Architecture	19
4.1	Overall Architecture	19
4.2	Frontend Architecture	20
4.3	Backend Architecture	20

4.4	Deployment Architecture	21
4.5	Authentication Flow	22
4.6	Data Flow Diagram	22
4.7	Route Optimization Flow	23
4.8	Driver Assignment Flow	23
4.9	Request Processing Flow	24
4.10	Module Interaction Diagram	25
5	Database Design	27
5.1	Entity-Relationship Diagram	27
5.2	Database Schema	28
5.3	Relationship Explanation	31
5.4	Design Rationale	32
6	Implementation	33
6.1	Authentication Module	33
6.2	Dashboard Module	34
6.3	Vehicle Module	34
6.4	Driver Module	35
6.5	Route Planner Module	37
6.6	OSRM Integration	38
6.7	CSV Upload Module	39
6.8	Analytics Module	40
6.9	Admin Module	40
6.10	Live Tracking Module	41
6.11	Fuel Estimation	41
6.12	Traffic Simulation	41
6.13	Backend Request Flow Summary	42

7	Algorithms	43
7.1	Nearest Neighbor Algorithm	43
7.2	Priority-Based Routing	44
7.3	Road Distance Calculation via OSRM	45
7.4	Haversine Fallback	45
7.5	ETA Calculation	46
7.6	Fuel Estimation	46
7.7	Vehicle Capacity Validation	47
7.8	Dijkstra's Algorithm and A*	47
7.9	Traffic Factor Simulation	48
8	Deployment	49
8.1	Containerization with Docker	49
8.2	Docker Compose	49
8.3	Production Deployment	50
8.4	Environment Configuration	51
8.5	Deployment Checklist	52
9	Testing	53
9.1	Unit Testing	53
9.2	API Testing	55
9.3	Performance Testing	56
9.4	Load Testing	60
9.5	Stress Testing	61
9.6	Security Testing	61
9.7	Authentication Testing	63
9.8	Database Testing	63
9.9	Integration Testing	63

10 Results	67
10.1 Performance	67
10.2 Deployment	67
10.3 Scalability	68
10.4 System Reliability	68
10.5 Summary of Results	69
11 Future Enhancements	70
11.1 AI-Based Route Prediction	70
11.2 Machine Learning for Traffic-Aware ETA	70
11.3 Electric Vehicle Route Optimization	70
11.4 Demand Prediction	71
11.5 Carbon Footprint Tracking	71
11.6 Additional Enhancements	71
12 Conclusion	73

List of Figures

4.1	Overall System Architecture	19
4.2	Frontend Architecture	20
4.3	Backend Architecture	21
4.4	Deployment Architecture	21
4.5	Authentication Flow	22
4.6	Data Flow Diagram – Route Creation	22
4.7	Route Optimization Decision Flow	23
4.8	Driver Assignment Flow	24
4.9	Request Processing Flow	25
4.10	Module Interaction Diagram	26
5.1	Entity-Relationship Diagram	27
8.1	Production Deployment Topology	51
9.1	k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics. . .	58
9.2	k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics. . .	58
9.3	k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics. . .	59
9.4	k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics. . .	59
9.5	k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics. . .	60
9.6	k6 load testing results	61

9.7	Administrator Dashboard displaying real-time fleet statistics, active routes, vehicle availability, fuel consumption, travel distance, and delivery metrics.	64
9.8	User Authentication page providing secure login access to the Route Optimization System.	64
9.9	Vehicle Management Dashboard displaying fleet information, vehicle capacity, fuel efficiency, availability, and management options. . . .	65
9.10	Driver Management Dashboard showing driver profiles, assigned vehicles, license details, availability status, and administrative controls. . . .	65
9.11	Analytics Dashboard presenting route status distribution and vehicle utilization through interactive charts for fleet performance analysis. . . .	66
9.12	Route Planner with an interactive Leaflet map displaying the optimized delivery route, delivery stops, travel distance, estimated travel time, and fuel consumption.	66

List of Tables

2.1	Functional Requirements	6
2.2	Non-Functional Requirements	8
3.1	Technology Stack Summary	18
5.1	User Table	28
5.2	Vehicle Table	29
5.3	Driver Table	29
5.4	Route Table	30
5.5	Stop Table	31
6.1	Authentication API Endpoints	34
6.2	Vehicle API Endpoints	35
6.3	Driver API Endpoints	37
6.4	Route API Endpoints	38
8.1	Backend Environment Variables	52
9.1	Unit Test Coverage Summary	54
9.2	Representative API Test Cases	55
9.3	Performance Test Results Summary	57
10.1	Summary of Results Against Objectives	69

CHAPTER 1

Introduction

1.1 Background

The logistics and delivery sector has grown rapidly over the last decade, driven largely by the expansion of e-commerce and food delivery platforms. As the number of daily deliveries handled by even moderately sized businesses has increased, the way delivery routes are planned has become a direct factor in operating cost. Industry studies on urban logistics consistently point to the "last mile" – the portion of a delivery's journey that takes a package from a local hub or warehouse to the customer's door – as the most expensive and time-consuming segment of the entire supply chain. This is primarily because the last mile involves many stops spread across a small geographic area, each with its own set of constraints such as delivery priority, vehicle capacity, and time windows.

Large logistics companies address this problem using dedicated route optimization platforms that are often built on top of commercial mapping APIs. These platforms are capable of computing optimized delivery sequences across hundreds of stops within seconds, taking real road networks, traffic conditions, and vehicle constraints into account. However, such platforms are usually priced on a per-request or per-vehicle basis, which makes them financially difficult to justify for smaller delivery operations, campus logistics teams, or student-run projects that still need reasonably accurate route planning.

This project was undertaken to explore whether a comparable, though naturally more modest, route optimization system could be built using entirely open-source components – specifically, by self-hosting a routing engine instead of relying on a paid mapping API. The result is the Smart Route Optimization System, a full-stack web application that plans, optimizes, and tracks last-mile delivery routes using real road data.

1.2 Problem Statement

Delivery operations that do not use a dedicated route optimization tool typically rely on one of two approaches: manual route planning by a dispatcher who is familiar with the local area, or naive straight-line distance calculations that do not account for the actual road network. Both approaches have clear shortcomings.

Manual planning does not scale well. As the number of vehicles and daily stops grows, it becomes increasingly difficult for a single dispatcher to mentally construct an efficient route, particularly one that also accounts for vehicle capacity limits and delivery priority. Straight-line distance calculations, on the other hand, are easy to compute but frequently produce misleading results – two delivery points that appear close together on a map may, in practice, require a long detour due to one-way streets, water bodies, or the layout of the road network.

At the same time, commercial routing APIs that solve this problem introduce a recurring cost that scales with usage, which is often disproportionate to the size of a small or medium delivery operation. This creates a gap for a system that can produce road-accurate route optimization without a per-request billing model.

1.3 Motivation

The motivation for this project came from observing this gap directly: the optimization logic required to sequence a set of delivery stops (nearest neighbor search, priority scheduling, capacity validation) is not, by itself, particularly expensive to compute. What makes commercial routing platforms valuable is primarily their access to accurate, continuously updated road network data and the infrastructure to query it quickly. The Open Source Routing Machine (OSRM) project makes exactly this kind of road network data queryable through a self-hosted service built on top of OpenStreetMap data, which is freely available and community maintained.

This made it possible to combine a custom-built optimization layer with a self-hosted routing engine, producing a system that behaves similarly to a commercial platform for the specific problem of last-mile delivery routing, without a recurring per-request cost. Building this system end to end – from the database schema through to a deployed, testable web application – also served as a practical exercise in full-stack engineering, covering backend API design, algorithm implementation, frontend dashboard development, and DevOps concerns such as containerized deployment.

1.4 Objectives

The project was carried out with the following specific objectives:

1. Design and implement a backend service capable of managing vehicles, drivers, and delivery routes through a role-based REST API.
2. Implement a route optimization engine that sequences delivery stops using a nearest-neighbor heuristic and an alternative priority-based strategy, while respecting vehicle capacity limits.
3. Integrate a self-hosted OSRM instance to replace straight-line distance calculations with real road distance, road travel time, and road geometry, while retaining a fallback path so that the system remains functional if the routing service becomes unavailable.
4. Build a responsive, role-aware frontend dashboard that allows dispatchers to plan and monitor routes, administrators to manage users and system-wide statistics, and drivers to view and update their assigned deliveries.
5. Provide analytics on fleet-wide distance, fuel consumption, travel time, and delivery completion, to give dispatchers visibility into overall operational performance.
6. Deploy the complete system using containerized infrastructure, with the frontend and backend hosted on separate, independently scalable platforms.
7. Test the system across functional, performance, load, and basic security dimensions, and document the results objectively.

1.5 Scope

The scope of this project covers the design and implementation of a complete route optimization platform for a single organization managing its own fleet of delivery vehicles and drivers. The system supports:

- Management of vehicles, including weight and volume capacity tracking.
- Management of drivers, including assignment to vehicles by email lookup.
- Creation of delivery routes from manually entered stops or a bulk CSV upload.

- Road-based route optimization using a self-hosted routing engine, with a straight-line fallback.
- Live tracking of vehicle positions using WebSocket-based updates.
- Role-based dashboards for Administrators, Dispatchers, and Drivers.
- An analytics dashboard summarizing operational metrics.

The scope explicitly does not include multi-organization (multi-tenant) support, dynamic re-routing based on live traffic conditions, or machine-learning-based demand forecasting – these are discussed as potential future enhancements in Chapter 11.

1.6 Expected Benefits

The system, as implemented, is expected to provide the following benefits to a delivery operation that adopts it:

- **Reduced route planning time** – Dispatchers no longer need to manually sequence delivery stops; the system generates an optimized order automatically.
- **More accurate distance and time estimates** – Because routes are computed against real road geometry rather than straight-line distance, the estimated distance, fuel consumption, and arrival time are considerably closer to what drivers will experience in practice.
- **No recurring routing API cost** – Since the routing engine is self-hosted, the system avoids the per-request billing model of commercial mapping APIs.
- **Operational visibility** – The analytics dashboard gives dispatchers and administrators a consolidated view of fleet utilization, completed and pending deliveries, and fuel usage, which is otherwise difficult to track manually.
- **Capacity-aware planning** – Vehicle weight capacity is validated automatically before a route is created, reducing the chance of a vehicle being assigned more load than it can carry.

The remainder of this report is organized as follows. Chapter 2 presents the functional and non-functional requirements. Chapter 3 discusses the technology choices made and the alternatives considered. Chapter 4 describes the system architecture in detail, supported by diagrams. Chapter 5 covers the database design. Chapter 6

walks through the implementation of each module. Chapter 7 explains the algorithms used for route optimization. Chapter 8 describes the deployment process. Chapter 9 documents the testing methodology and results. Chapter 10 discusses overall results. Chapter 11 outlines future enhancements, and Chapter 12 concludes the report.

CHAPTER 2

Requirement Analysis

Before implementation began, the project requirements were separated into functional requirements (what the system must do), non-functional requirements (how the system must behave), user requirements (what each role needs from the system), business requirements (why the system is being built), and a set of constraints and assumptions that shaped the design.

2.1 Functional Requirements

Table 2.1 lists the functional requirements identified for the system, grouped by module.

Table 2.1: Functional Requirements

ID	Requirement
FR-01	Users shall be able to register and log in using an email and password, with the account assigned one of three roles: Administrator, Dispatcher, or Driver.
FR-02	The system shall issue a JSON Web Token (JWT) upon successful login, which is required for all subsequent authenticated requests.
FR-03	Administrators and Dispatchers shall be able to create, update, and delete vehicle records, including plate number, type, fuel efficiency, and capacity attributes (weight and volume).
FR-04	Administrators and Dispatchers shall be able to create driver profiles by looking up an existing user account by email address and linking it to a vehicle.
FR-05	The system shall reject a driver assignment request if no user account exists with the given email, returning a clear error message.

ID	Requirement
FR-06	Dispatchers shall be able to create a delivery route by specifying a depot location and a set of delivery stops, either entered manually or uploaded as a CSV file.
FR-07	The system shall optimize the order of delivery stops using a selectable strategy: nearest-neighbor or priority-first.
FR-08	The system shall compute the route using real road distance and travel time obtained from a self-hosted OSRM instance.
FR-09	If the OSRM service is unavailable, the system shall automatically fall back to a straight-line (Haversine) distance calculation so that route creation is not blocked.
FR-10	The system shall reject route creation if the total demand of the delivery stops exceeds the assigned vehicle's available capacity.
FR-11	The system shall display the optimized route on an interactive map, drawing the actual road path when available.
FR-12	Drivers shall be able to view their currently assigned route, mark individual stops as visited, and update the route status (start, complete).
FR-13	The system shall broadcast live vehicle location updates to connected clients using WebSockets.
FR-14	The system shall provide an analytics dashboard showing total distance, total travel time, completed and pending deliveries, fuel consumption, and route status breakdown.
FR-15	Administrators shall be able to view and manage all registered users and their roles.

2.2 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

ID	Requirement
NFR-01	Performance – Route optimization for a typical route of 8–12 stops should complete within a few seconds under normal load, including the round trip to the OSRM service.
NFR-02	Availability – Core route creation should degrade gracefully rather than fail outright if the routing engine is temporarily unreachable.
NFR-03	Security – All state-changing API endpoints shall require a valid JWT, and role-sensitive endpoints shall additionally enforce role-based authorization.
NFR-04	Usability – The dashboard shall be responsive and usable on both desktop and mobile viewports, since drivers are expected to access their assignments primarily from a phone.
NFR-05	Maintainability – The backend and frontend codebases shall be written in TypeScript with a modular structure separating routes, controllers, services, and algorithms.
NFR-06	Portability – The system shall be deployable using Docker Compose so that the full stack, including the routing engine, can be reproduced consistently across environments.
NFR-07	Scalability – The backend shall be stateless with respect to application logic, so that additional instances can be run behind a load balancer if request volume increases.

2.3 User Requirements

The system is designed around three distinct user roles, each with different needs.

Administrator. The administrator needs full visibility into the system: the ability to view and manage all registered users, change user roles, and view system-wide statistics. Administrators are also able to perform every action available to a dispatcher.

Dispatcher. The dispatcher is the primary operator of the system on a day-to-day basis. A dispatcher needs to add and update vehicles, create driver profiles and

assign them to vehicles, upload or manually enter delivery stops, generate optimized routes, and monitor route progress and fleet analytics.

Driver. A driver needs a simple, mobile-friendly view of their current assignment: the sequence of stops to visit, the remaining distance and estimated arrival time, and a way to mark individual stops as completed and update the overall route status.

2.4 Business Requirements

From a business perspective, the system needed to satisfy the following higher-level goals, which guided several of the functional and non-functional requirements above:

- Avoid recurring third-party routing API costs by using a self-hosted routing engine.
- Reduce the manual effort currently required to plan delivery routes.
- Provide a single source of truth for vehicle, driver, and route data instead of relying on spreadsheets.
- Support gradual adoption – the system should remain usable (via the fallback path) even before the routing engine is fully configured with regional map data.

2.5 Constraints

- The routing engine (OSRM) requires pre-processed OpenStreetMap data for the region of operation; this data must be prepared before the routing engine can serve road-accurate results.
- Free-tier deployment platforms (used for Render and Vercel in this project) impose limits on compute resources and may introduce cold-start latency after periods of inactivity.
- The project timeline did not allow for dynamic re-routing based on live traffic data; the "traffic factor" used in ETA calculations is a configurable multiplier rather than a live traffic feed.

2.6 Assumptions

- Each vehicle is operated by, at most, one driver at a time.
- Delivery stops within a single route are all serviced by a single vehicle in one continuous trip starting and ending at the depot.
- Users creating driver profiles have already registered the corresponding driver's user account through the standard registration flow.
- The organization deploying the system operates within a single geographic region small enough to be covered by a single OSRM dataset.

CHAPTER 3

Technology Stack

This chapter describes each major technology used in the system, the reasoning behind its selection, and the alternatives that were considered before making that choice. The technology stack is split into frontend, backend, routing engine, and deployment infrastructure.

3.1 Frontend Technologies

3.1.1 React with TypeScript

React was chosen as the frontend library because of its component-based architecture, which maps naturally onto the dashboard's structure of reusable elements such as vehicle cards, route tables, and status badges. TypeScript was used throughout the frontend (and backend) codebase to catch type errors at compile time rather than at runtime, which was particularly valuable given the number of data structures shared between the API responses and the UI components – for instance, the shape of a `RouteRecord`, including its optimized stops, geometry, and computed metrics, is used consistently across the route planner, route details, and driver panel components.

Alternatives considered: Vue.js and Angular were both considered. Vue was ruled out mainly due to a smaller ecosystem of ready-made component libraries relevant to this project's needs (map integration, chart integration). Angular was ruled out because its heavier framework conventions and steeper learning curve were not justified for a single-developer project of this scope; React's more incremental, library-based approach allowed faster iteration.

3.1.2 Vite

Vite was used as the frontend build tool and development server. Compared to older bundlers, Vite's use of native ES modules during development results in near-

instant server start and hot module replacement, which noticeably sped up the development feedback loop given the size of the component tree in this project.

Alternatives considered: Create React App (CRA) was the more familiar option but has since fallen out of active maintenance and has noticeably slower rebuild times as a project grows. Webpack configured manually was also considered but rejected due to the additional configuration overhead that Vite avoids by default.

3.1.3 Tailwind CSS

Tailwind CSS was used for styling the entire frontend. Its utility-class approach made it possible to build a consistent design system (spacing, color palette, shadows, border radii) directly in component markup without maintaining a large separate stylesheet, and its built-in dark mode support (via the `class` strategy) was used to implement the application's light/dark theme toggle.

Alternatives considered: Plain CSS Modules and styled-components were both considered. Both were rejected primarily because maintaining consistent spacing and color values across dozens of components was judged to be more error-prone without a shared utility vocabulary such as Tailwind's design tokens.

3.1.4 React Router

React Router was used to implement client-side routing across the dashboard's pages (login, register, dashboard, route planner, vehicle management, driver panel, live tracking, analytics, admin panel), including role-based route protection through a `ProtectedRoute` wrapper component.

3.1.5 Axios

Axios was used as the HTTP client for all API communication. Its support for request and response interceptors was used to automatically attach the JWT to outgoing requests and to handle 401 responses globally by redirecting to the login page.

3.1.6 React Query

React Query (`react-query`) was used to manage server state – fetching, caching, and invalidating data such as the vehicle list, driver list, and route list – instead of manually managing loading and error state with `useState/useEffect` for every API call. This significantly reduced boilerplate and ensured that, for example, the vehicle list automatically refreshes after a vehicle is created or deleted.

Alternatives considered: Redux Toolkit Query was considered, but React Query was preferred because the project’s state is predominantly server state (data fetched from the API) rather than complex client-only state, which is exactly the use case React Query is designed around.

3.1.7 React Hook Form

React Hook Form was used for all forms in the application (login, registration, vehicle form, driver form, route planner). It was chosen for its minimal re-render behavior and straightforward integration with TypeScript, which was useful given the number of forms with typed payloads matching the backend’s expected request bodies.

3.1.8 Leaflet and React Leaflet

Leaflet, through its React bindings, was used to render the interactive maps used throughout the application – the route planner map, the route details map, and the live tracking map. Leaflet was chosen specifically because it is a free, open-source mapping library that does not require an API key or usage-based billing, which aligned with the project’s broader goal of avoiding recurring third-party costs. It also integrates cleanly with OpenStreetMap tiles, which pairs naturally with OSRM, since both are built on the same underlying map data.

Alternatives considered: Mapbox GL JS offers more advanced rendering (vector tiles, 3D terrain) but requires an API key and has usage-based pricing beyond a free tier. Google Maps JavaScript API was ruled out entirely, consistent with the project’s decision to avoid Google Maps dependencies throughout the stack.

3.1.9 Socket.IO Client

The Socket.IO client library was used to receive real-time vehicle location broadcasts from the backend, powering the live tracking page.

3.1.10 Chart.js

Chart.js (via `react-chartjs-2`) was used to render the analytics dashboard's charts – a doughnut chart for route status breakdown and a bar chart for vehicle utilization. It was chosen for being lightweight and having a straightforward declarative API that did not require learning a larger visualization grammar for the relatively simple charts required here.

3.2 Backend Technologies

3.2.1 Node.js and Express

Node.js was selected as the backend runtime primarily for consistency – using JavaScript/TypeScript across both the frontend and backend allowed type definitions (such as the shape of a route or vehicle) to be reasoned about consistently, even though they are maintained separately in each codebase. Express was used as the web framework due to its minimal, unopinionated structure, which made it straightforward to organize the backend into clearly separated route, controller, service, and algorithm layers.

Alternatives considered: A Python backend using FastAPI was considered, particularly given Python's strong ecosystem for numerical and graph algorithms. Express was ultimately preferred because it kept the entire stack in one language, simplifying shared type definitions and reducing context-switching during development.

3.2.2 TypeScript

TypeScript was used on the backend for the same reasons as the frontend – compile-time type checking – with the added benefit that Prisma generates fully typed database client code from the schema definition, which TypeScript can then check against the rest of the application code.

3.2.3 Prisma ORM

Prisma was used as the Object-Relational Mapping (ORM) layer between the Express backend and the PostgreSQL database. Prisma's schema file acts as a single source of truth for the database structure, from which it generates a fully typed query client and manages schema migrations. This was particularly useful as the schema evolved during development – for example, when vehicle capacity fields and route geometry storage were added partway through the project, Prisma's migration tooling made it straightforward to apply these changes without hand-writing SQL migration scripts.

Alternatives considered: TypeORM and Sequelize were both considered. Prisma was preferred for its stronger TypeScript integration (fully generated types rather than decorator-based type annotations) and its more predictable migration workflow.

3.2.4 PostgreSQL

PostgreSQL was chosen as the relational database. The system's data – users, vehicles, drivers, routes, and stops – is inherently relational, with clear foreign key relationships between them, which fits naturally into a relational schema. PostgreSQL was preferred over other relational databases for its maturity, strong support for JSON column types (used to store route geometry), and its wide availability as a managed service across deployment platforms.

Alternatives considered: MongoDB was considered, but the strongly relational nature of the data (a route belongs to a vehicle and a driver, which belong to a user, and consists of many ordered stops) made a relational database a more natural fit than a document store.

3.2.5 Redis

Redis was included in the stack for caching and for its planned role as a backing store for background job queues (via BullMQ) in later iterations of the vehicle and route services. In the current implementation, Redis is primarily used for caching and is provisioned as part of the Docker Compose stack so that it is available for these use cases as the system evolves.

3.2.6 JWT and Bcrypt

JSON Web Tokens (JWT) were used to implement stateless authentication: after a successful login, the backend issues a signed token containing the user's ID and role, which the frontend attaches to subsequent requests. Bcrypt was used to hash user passwords before storing them, ensuring that plaintext passwords are never persisted in the database.

3.2.7 Socket.IO (Server)

Socket.IO was used on the backend to broadcast vehicle location updates to all connected clients, powering the live tracking feature described in Section 6.10.

3.2.8 Multer

Multer was used to handle multipart/form-data file uploads on the backend, specifically for the CSV upload feature that allows dispatchers to bulk import delivery stops.

3.3 Routing Engine

3.3.1 OSRM (Open Source Routing Machine)

OSRM is the routing engine at the center of this project's core differentiator. It is an open-source C++ routing engine that pre-processes OpenStreetMap data into a format that can answer shortest-path and distance-matrix queries very quickly. This project uses two of OSRM's HTTP API endpoints:

- `/table` – returns a full pairwise distance and duration matrix for a set of coordinates, used to drive the nearest-neighbor stop selection.
- `/route` – returns the exact road geometry, total distance, total duration, and per-leg distance/duration for an ordered sequence of coordinates, used to produce the final route metrics and the polyline drawn on the map.

Alternatives considered: The Google Maps Directions API and Distance Matrix API were the most obvious alternatives and would have required no self-hosted

infrastructure, but both are billed per request beyond a limited free quota, which conflicts directly with this project's goal of avoiding recurring routing costs. Graph-Hopper, another open-source routing engine, was also considered; OSRM was ultimately selected due to its widespread adoption, extensive documentation, and the availability of an official Docker image, which simplified the Docker Compose deployment described in Chapter 8.

3.4 Deployment Technologies

3.4.1 Docker and Docker Compose

Docker was used to containerize the backend, frontend, and OSRM services, and Docker Compose was used to orchestrate them together with PostgreSQL and Redis as a single reproducible stack. This was particularly valuable for the OSRM service, which depends on pre-processed map data files; packaging it as a service within the same Compose file as the rest of the application ensured that the entire system, including the routing engine, could be started with a single command in any environment.

3.4.2 Render

Render was used to host the backend service in production. It was chosen for its straightforward Docker-based deployment model and its provision of a managed PostgreSQL database, which removed the need to separately administer database infrastructure.

3.4.3 Vercel

Vercel was used to host the frontend. Its tight integration with Git-based deployment workflows (automatic deployment on push) and its strong support for Vite-based single-page applications made it a natural fit for the frontend deployment.

Alternatives considered: Netlify was considered as an alternative to Vercel for the frontend, and Railway and Fly.io were considered as alternatives to Render for the backend. The final choices were driven mainly by familiarity and the generous free tiers available on Render and Vercel at the time of deployment, which was an important consideration given the project's cost constraints described in Chapter 2.

3.5 Summary

Table 3.1 summarizes the complete technology stack used in the system.

Table 3.1: Technology Stack Summary

Layer	Technology
Frontend Framework	React 18 with TypeScript
Build Tool	Vite
Styling	Tailwind CSS
Routing	React Router
HTTP Client	Axios
Server State Management	React Query
Form Handling	React Hook Form
Maps	Leaflet / React Leaflet
Real-Time Client	Socket.IO Client
Charts	Chart.js
Backend Runtime	Node.js
Backend Framework	Express.js with TypeScript
ORM	Prisma
Database	PostgreSQL
Cache / Queue Backing Store	Redis
Authentication	JWT + Bcrypt
Real-Time Server	Socket.IO
File Uploads	Multer
Routing Engine	OSRM (Open Source Routing Machine)
Containerization	Docker, Docker Compose
Frontend Hosting	Vercel
Backend Hosting	Render

CHAPTER 4

System Architecture

This chapter describes the architecture of the Smart Route Optimization System at multiple levels of detail: the overall system architecture, the internal structure of the frontend and backend, the deployment topology, and the flow of data through several of the system's key operations.

4.1 Overall Architecture

The system follows a fairly conventional three-tier architecture, with a clear separation between the presentation layer (the React frontend), the application layer (the Express backend, including the optimization algorithms), and the data layer (PostgreSQL and Redis). The routing engine (OSRM) sits alongside the application layer as an external service that the backend queries during route optimization. Figure 4.1 shows this structure.

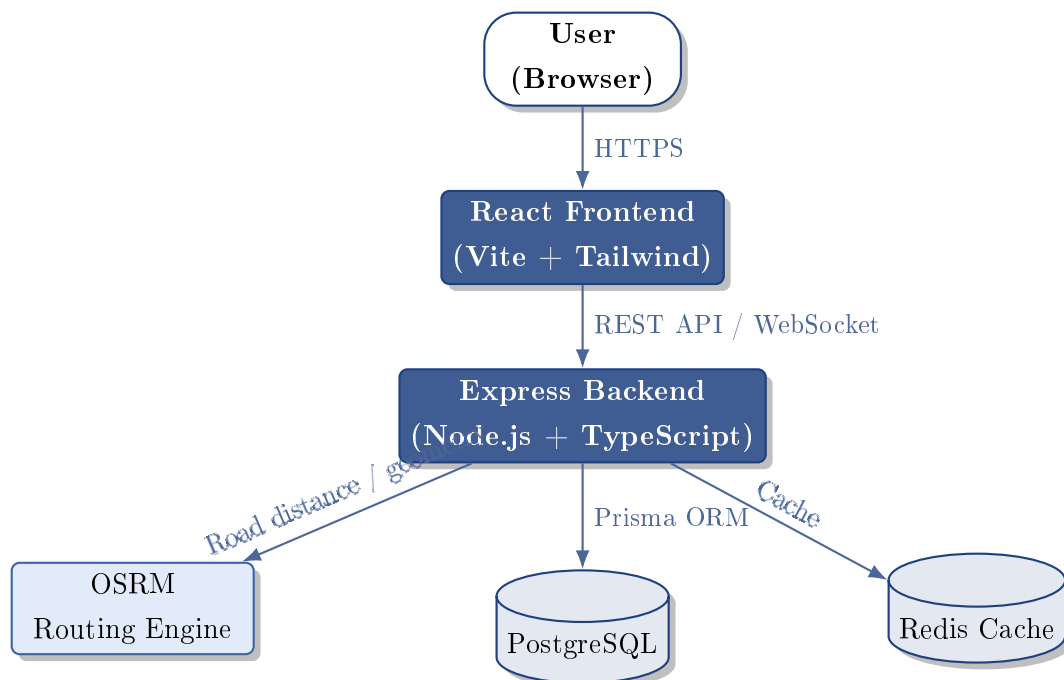


Figure 4.1: Overall System Architecture

The frontend never communicates directly with the database, the routing engine, or Redis; all such communication is mediated by the backend, which enforces authentication, authorization, and business rules (such as vehicle capacity validation) before any data is persisted.

4.2 Frontend Architecture

The frontend is organized around three layers: pages (route-level components corresponding to each URL), feature components (grouped by module, such as `vehicles`, `drivers`, `routes`, and `analytics`), and a shared set of design-system primitives (`Button`, `Card`, `Modal`, `Badge`, and similar). Server state is managed through React Query hooks, and two React Context providers – `AuthContext` and `SocketContext` – supply authentication state and the live WebSocket connection to any component that needs them. Figure 4.2 illustrates this layered structure.

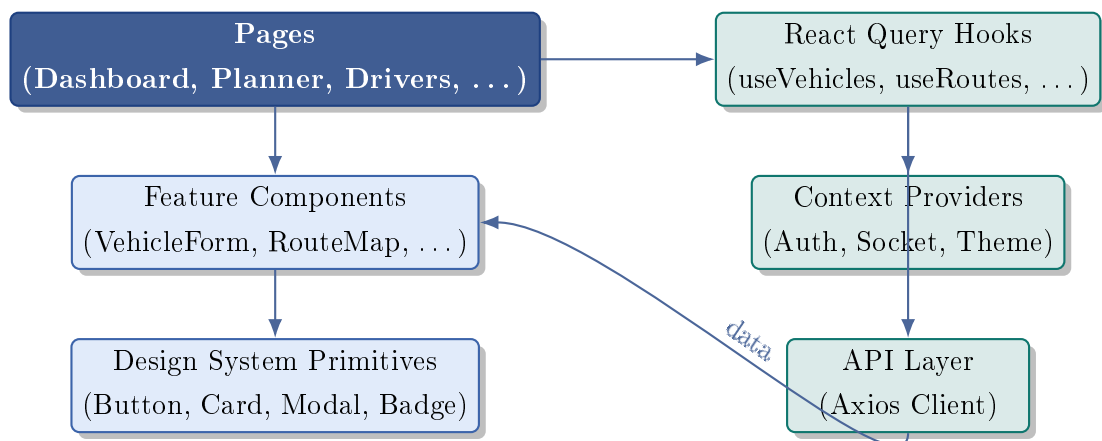


Figure 4.2: Frontend Architecture

4.3 Backend Architecture

The backend follows a layered structure that separates HTTP concerns from business logic. Incoming requests pass through Express middleware (CORS, JSON parsing, authentication) before reaching a route definition, which delegates to a controller. Controllers are intentionally thin – they parse the request, call a service function, and shape the HTTP response. Services contain the actual business logic and are the only layer that interacts with Prisma (the database) or external services such as OSRM. The optimization algorithms are implemented as a separate mod-

ule, consumed by the route service, so that the algorithm implementations remain independent of Express and Prisma entirely and can be unit tested in isolation. Figure 4.3 shows this layering.

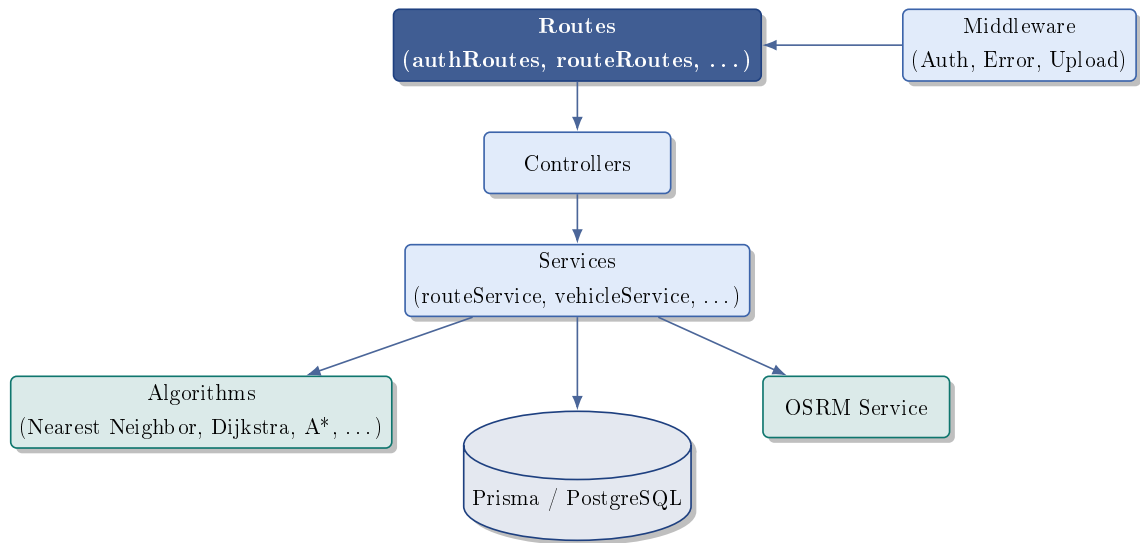


Figure 4.3: Backend Architecture

4.4 Deployment Architecture

In production, the frontend and backend are deployed independently. The frontend is a static build served from Vercel’s edge network. The backend, OSRM, PostgreSQL, and Redis are deployed as containerized services, with the backend and database hosted on Render and OSRM run alongside it using the Docker Compose configuration described in Chapter 8. Figure 4.4 shows this deployment topology.

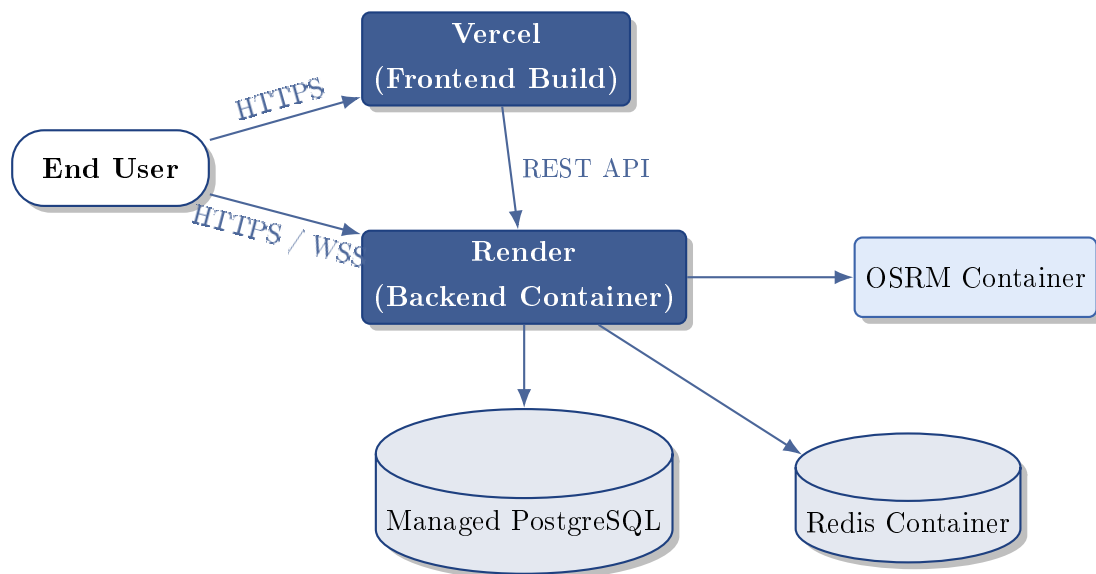


Figure 4.4: Deployment Architecture

4.5 Authentication Flow

Authentication is implemented using JWTs. Figure 4.5 shows the sequence of steps involved in logging in and making an authenticated request.

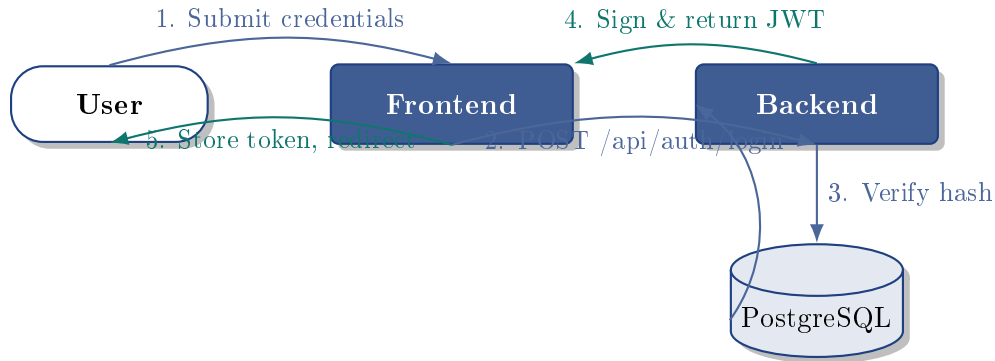


Figure 4.5: Authentication Flow

Once issued, the token is stored in the browser and attached to every subsequent request as an **Authorization: Bearer <token>** header. An Axios request interceptor on the frontend handles this automatically, and an Express middleware (**authMiddleware**) verifies the token on the backend before allowing the request to reach a controller. If the token is missing, invalid, or expired, the backend responds with a 401 status, and the frontend's response interceptor clears the stored session and redirects the user to the login page.

4.6 Data Flow Diagram

Figure 4.6 presents a simplified data flow diagram for the core route creation operation, showing how data moves between the dispatcher, the frontend, the backend's route service, the optimization algorithms, the OSRM engine, and the database.

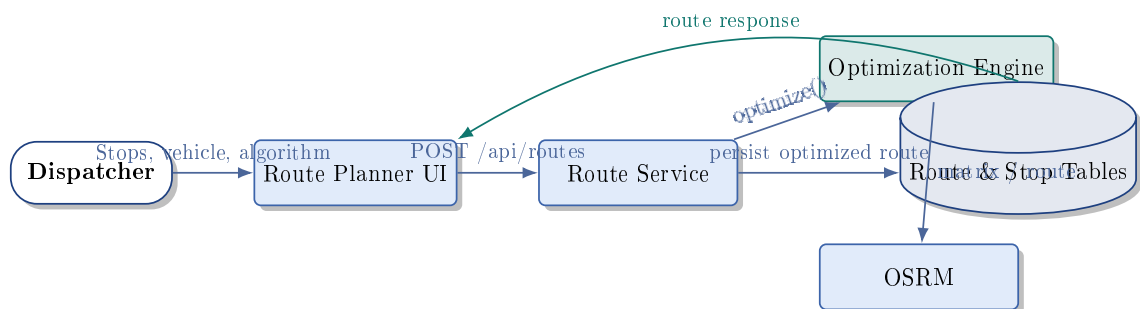


Figure 4.6: Data Flow Diagram – Route Creation

4.7 Route Optimization Flow

Figure 4.7 shows the decision flow inside the optimization service when a route creation request is processed. Vehicle capacity is validated first, since it is the cheapest check and should fail fast before any external service is called. The system then attempts road-based optimization through OSRM; if that call fails for any reason (timeout, network error, or an OSRM error response), the algorithm falls back to the Haversine-based calculation rather than failing the request outright.

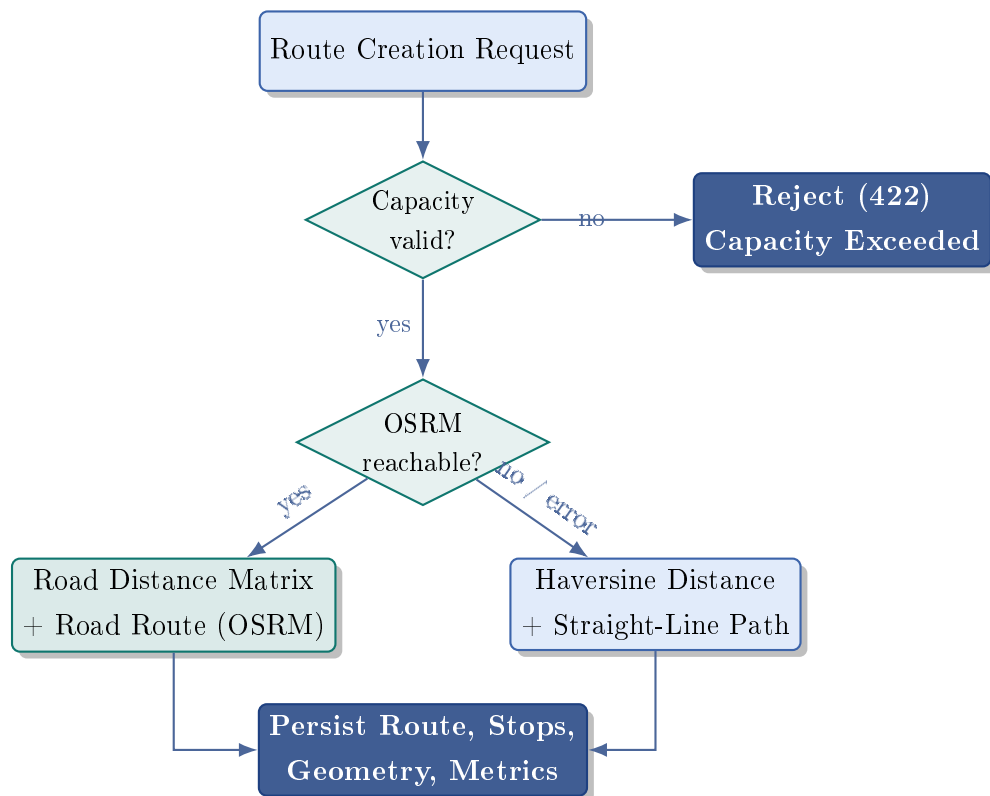


Figure 4.7: Route Optimization Decision Flow

4.8 Driver Assignment Flow

Driver assignment is performed by email lookup rather than by a raw database identifier, so that a dispatcher does not need to know a driver's internal user ID. Figure 4.8 shows this flow, including the explicit "Driver not found" rejection path when the supplied email does not match any registered account.

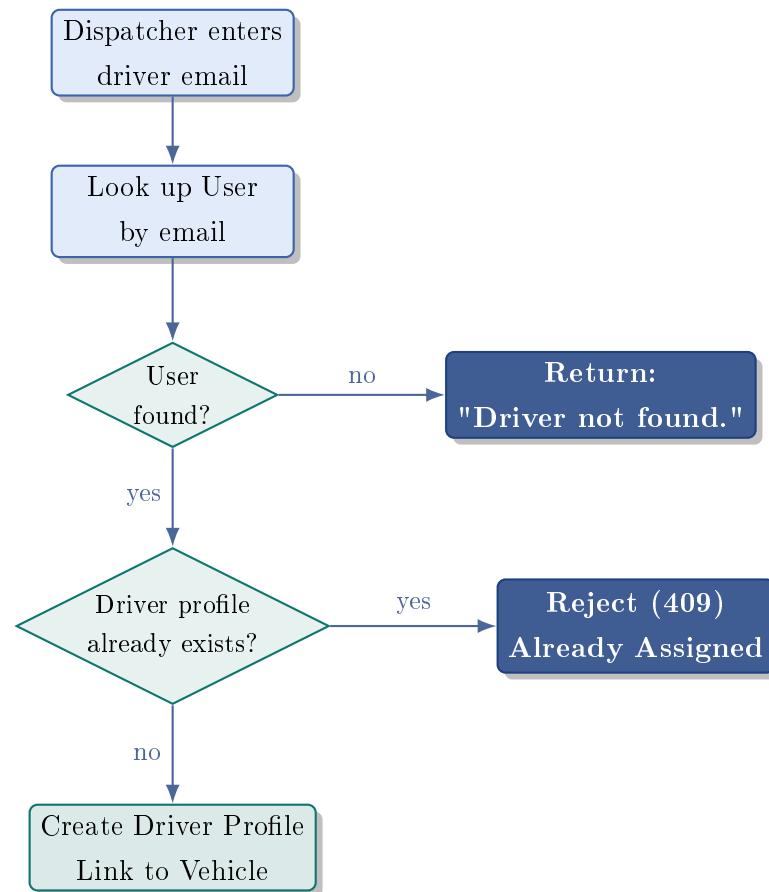
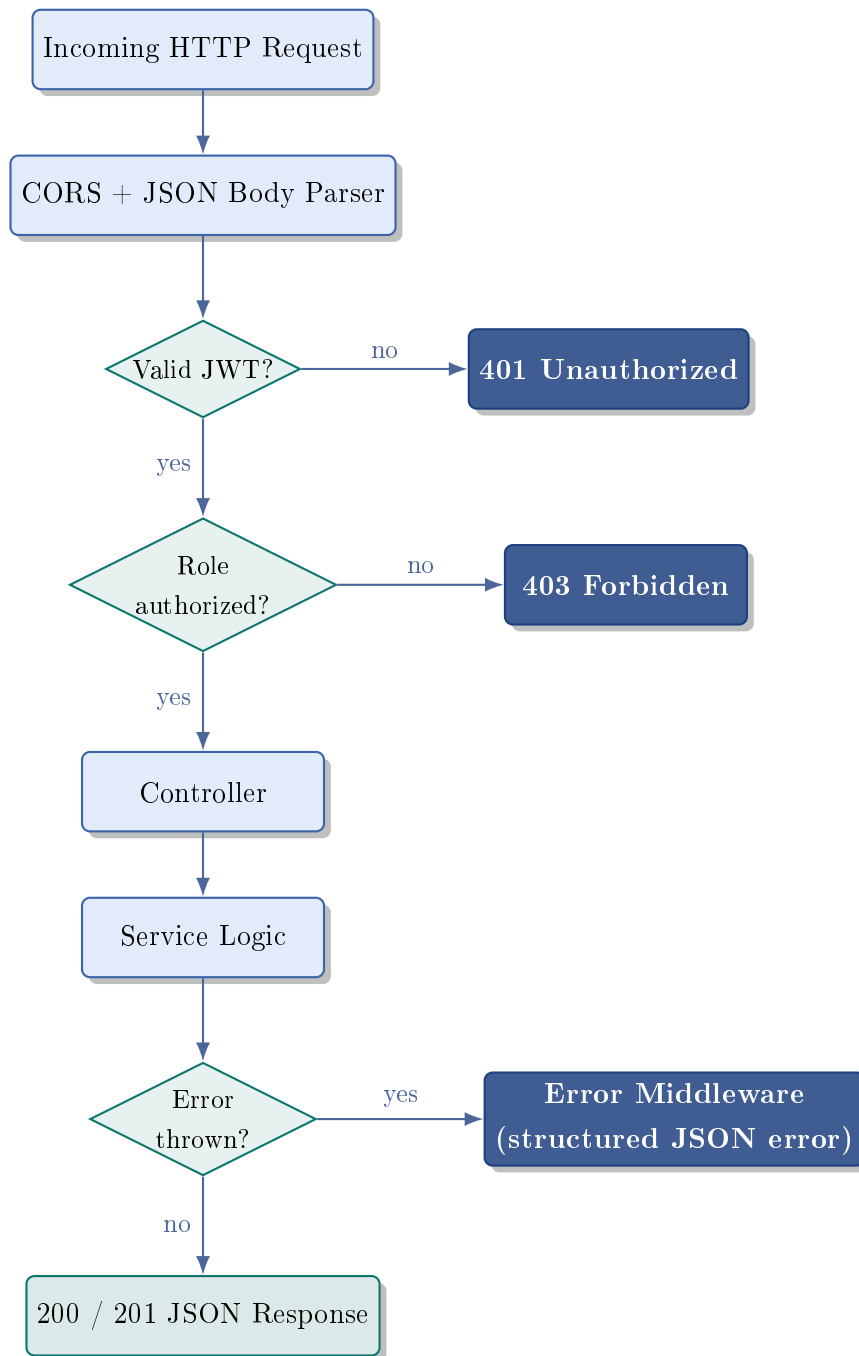


Figure 4.8: Driver Assignment Flow

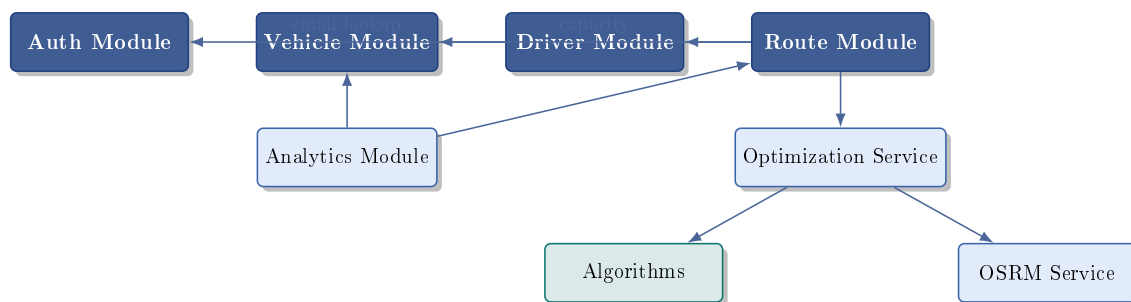
4.9 Request Processing Flow

Figure 4.9 shows the general path taken by every authenticated API request, illustrating where middleware, authorization, and error handling fit into the process.

**Figure 4.9:** Request Processing Flow

4.10 Module Interaction Diagram

Figure 4.10 shows how the main backend modules depend on one another. Arrows indicate a "depends on" or "calls into" relationship. Notably, the algorithm modules (nearest neighbor, priority scheduling, fuel estimation, ETA calculation, capacity validation) have no outgoing dependencies on Express, Prisma, or OSRM, which keeps them easy to unit test in isolation, as discussed in Chapter 9.

**Figure 4.10:** Module Interaction Diagram

CHAPTER 5

Database Design

The database schema is managed through Prisma and implemented on PostgreSQL. This chapter presents the entity-relationship model, the resulting schema, and an explanation of the relationships between tables.

5.1 Entity-Relationship Diagram

The database consists of five core entities: **User**, **Vehicle**, **Driver**, **Route**, and **Stop**. A **User** may have at most one **Driver** profile. A **Vehicle** may be linked to at most one **Driver** at a time, and may be associated with many **Route** records over its lifetime. A **Route** belongs to one **Vehicle** and, optionally, one **Driver**, and consists of many ordered **Stop** records. Figure 5.1 shows this structure.

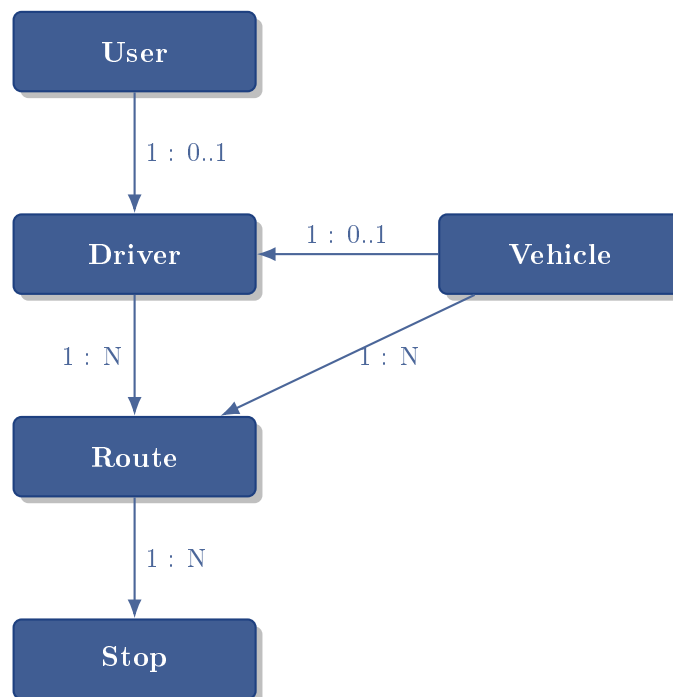


Figure 5.1: Entity-Relationship Diagram

5.2 Database Schema

The following tables describe the structure of each entity as implemented in the Prisma schema and reflected in the PostgreSQL database.

Table 5.1: User Table

Column	Type	Notes
id	UUID	Primary key
name	String	
email	String	Unique
passwordHash	String	Bcrypt hash, never stored in plaintext
role	Enum	ADMIN, DISPATCHER, DRIVER
createdAt	DateTime	
updatedAt	DateTime	

Table 5.2: Vehicle Table

Column	Type	Notes
id	UUID	Primary key
plateNumber	String	Unique
type	String	
capacityKg	Float	Legacy weight capacity field
fuelEfficiency	Float	km per litre
maxWeight	Float (nullable)	Falls back to capacityKg if unset
maxVolume	Float (nullable)	Stored, not yet enforced
currentLoad	Float	Default 0, tracks committed weight
status	Enum	AVAILABLE, ON_ROUTE, MAINTENANCE
latitude, longitude	Float (nullable)	Last known live position
createdAt, updatedAt	DateTime	

Table 5.3: Driver Table

Column	Type	Notes
id	UUID	Primary key
userId	UUID	Foreign key to User, unique
phone	String	
licenseNo	String	
vehicleId	UUID (nullable)	Foreign key to Vehicle, unique
available	Boolean	Default true
createdAt	DateTime	

Table 5.4: Route Table

Column	Type	Notes
id	UUID	Primary key
name	String	
vehicleId	UUID (nullable)	Foreign key to Vehicle
driverId	UUID (nullable)	Foreign key to Driver
status	Enum	PLANNED, IN_PROGRESS, COMPLETED, CANCELLED
algorithm	String	nearest-neighbor / priority-first
totalDistanceKm	Float	Road distance (or fallback distance)
totalTravelTime	Float (nullable)	Pure travel minutes, excludes dwell time
geometry	JSON (nullable)	[lng, lat] road path from OSRM
estimatedFuel	Float	Litres
etaMinutes	Float	Travel time + stop dwell overhead
createdAt, updatedAt	DateTime	

Table 5.5: Stop Table

Column	Type	Notes
id	UUID	Primary key
routeId	UUID	Foreign key to Route (cascade delete)
sequence	Integer	Visiting order within the route
label	String	
latitude, longitude	Float	
demandKg	Float	Default 0
priority	Integer	Default 1
visited	Boolean	Default false
distanceFromPrevious	Float	Road distance from the previous stop
durationFromPrevious	Float	Road travel time from the previous stop
createdAt	DateTime	

5.3 Relationship Explanation

User – Driver. A one-to-zero-or-one relationship. Every driver profile is linked to a user account, but not every user account has a driver profile – only accounts that have been explicitly registered as drivers by an administrator or dispatcher. This relationship is looked up by email during driver creation, as described in Section 6.4.

Vehicle – Driver. A one-to-zero-or-one relationship in the current implementation: a vehicle can have at most one driver assigned to it at a time, and a driver can be assigned to at most one vehicle. This was a deliberate simplification, since the target use case is fleets where each vehicle has a consistent, dedicated driver.

Vehicle – Route. A one-to-many relationship. A single vehicle can be used across many routes over time (though only one active route at a time in practice, since the vehicle’s status moves to **ON_ROUTE** when a route is created and back to **AVAILABLE** on completion or cancellation).

Driver – Route. A one-to-many relationship, analogous to the vehicle relationship above.

Route – Stop. A one-to-many relationship with cascading delete – if a route is deleted, its stops are deleted along with it. Each stop stores its own sequence number, demand weight, priority, and the road distance/duration from the previous stop, which allows the driver panel to compute remaining distance and estimated arrival time without recomputing the entire route.

5.4 Design Rationale

Two schema decisions are worth calling out specifically, since they were added after the initial schema design in response to the road-routing upgrade described in Section 6.6.

The **geometry** column on **Route** stores the OSRM-returned polyline (or the straight-line fallback path) as JSON directly in PostgreSQL, rather than in a separate table or file store. This was a pragmatic choice: the geometry is always read as a whole alongside the rest of the route record, so there was no benefit to normalizing it further, and PostgreSQL’s native JSON column type made this straightforward through Prisma.

The **distanceFromPrevKm** and **durationFromPrevMin** columns on **Stop** store the per-leg distance and duration between consecutive stops, taken directly from OSRM’s **legs** array in the **/route** response. Storing this at the stop level, rather than only storing the route’s total distance and duration, made it possible to compute a driver’s remaining distance and estimated arrival time by summing the values for unvisited stops, without issuing another OSRM request every time the driver panel is opened.

CHAPTER 6

Implementation

This chapter describes how each module of the system was implemented, covering the backend flow, the frontend flow, the relevant API endpoints, and the way each module interacts with the database. Code listings are used where they help illustrate a specific implementation detail; the listings are simplified where necessary for readability but reflect the actual logic used in the system.

6.1 Authentication Module

The authentication module handles registration, login, and profile retrieval. On the backend, passwords are hashed using Bcrypt before being stored, and a signed JWT containing the user's ID and role is issued on successful login or registration. Listing 6.1 shows the core of the login flow.

```
1 export async function loginUser(input: LoginInput) {
2   const user = await prisma.user.findUnique({ where: { email:
    input.email } });
3   if (!user) throw new AppError("Invalid credentials", 401);
4
5   const valid = await comparePassword(input.password, user.
    passwordHash);
6   if (!valid) throw new AppError("Invalid credentials", 401);
7
8   const token = signToken({ userId: user.id, role: user.role });
9   return { token, user: { id: user.id, name: user.name, email:
    user.email, role: user.role } };
10 }
```

Listing 6.1: Login flow in authService.ts

On the frontend, an `AuthContext` provider stores the current user and token in React state, persists them to `localStorage` so that a session survives a page refresh, and exposes `login`, `register`, and `logout` functions to the rest of the application. An Axios request interceptor attaches the stored token to every outgoing request, and a response interceptor clears the session and redirects to the login page if a

request returns a 401 status, which happens when the token is missing, invalid, or expired.

Table 6.1 lists the authentication API endpoints.

Table 6.1: Authentication API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Create a new user account
POST	/api/auth/login	Authenticate and receive a JWT
GET	/api/auth/me	Return the authenticated user's profile

6.2 Dashboard Module

The dashboard is the landing page after login and gives an at-a-glance summary of fleet-wide statistics: total and active routes, total and available vehicles, total drivers, total distance travelled, total travel time, completed and pending deliveries, and the next scheduled ETA across all planned or in-progress routes. This data is served by a single `GET /api/analytics/dashboard` endpoint, backed by a set of aggregate Prisma queries in `analyticsService.ts` that run in parallel using `Promise.all`, keeping the endpoint fast even as the number of routes and stops grows. Completed and pending delivery counts are computed by counting `Stop` rows where `visited` is true or false respectively, across all routes.

6.3 Vehicle Module

The vehicle module supports full CRUD (create, read, update, delete) operations on vehicles, along with the capacity-related fields introduced during the capacity enforcement upgrade: `maxWeight`, `maxVolume`, and `currentLoad`. A vehicle's available capacity for new deliveries is computed as its `maxWeight` (falling back to the legacy `capacityKg` field if `maxWeight` is not set) minus its `currentLoad`, as shown in Listing 6.2.

```
1 export function getAvailableCapacityKg(vehicle: {
2   maxWeight: number | null;
3   capacityKg: number;
4   currentLoad: number;
```

```

5  }): number {
6    const limit = vehicle.maxWeight ?? vehicle.capacityKg;
7    return Math.max(limit - vehicle.currentLoad, 0);
8  }

```

Listing 6.2: Available capacity calculation in vehicleService.ts

This function is called by the route service before optimization begins, so that a route can never be created if the combined demand of its stops would exceed the vehicle’s remaining capacity. When a route is created, the vehicle’s `currentLoad` is incremented by the total demand of the route’s stops, and its status changes to `ON_ROUTE`. When the route is later marked `COMPLETED` or `CANCELLED`, the vehicle’s `currentLoad` is decremented by the same amount and its status reverts to `AVAILABLE`.

On the frontend, the Vehicle Management page displays vehicles as cards in a responsive grid, each showing a load-utilization bar computed from `currentLoad` against the effective capacity limit, with the bar turning red once utilization crosses ninety percent. Vehicles are created and edited through a modal form built with React Hook Form.

Table 6.2: Vehicle API Endpoints

Method	Endpoint	Description
POST	/api/vehicles	Create a vehicle
GET	/api/vehicles	List all vehicles
GET	/api/vehicles/:id	Retrieve a single vehicle
PUT	/api/vehicles/:id	Update a vehicle
DELETE	/api/vehicles/:id	Delete a vehicle
PATCH	/api / vehicles/ id/location	: Update a vehicle’s live position

6.4 Driver Module

The driver module was changed during development so that drivers are assigned by looking up an existing user account by email address, instead of requiring the dispatcher to know that user’s raw database identifier. Listing 6.3 shows the corresponding backend logic.

```
1 export async function createDriver(input: DriverInput) {
2   const user = await prisma.user.findUnique({ where: { email:
3     input.email } });
4   if (!user) {
5     throw new AppError("Driver not found.", 404);
6   }
7   const existing = await prisma.driver.findUnique({ where: {
8     userId: user.id } });
9   if (existing) {
10    throw new AppError("Driver profile already exists for this
11      user", 409);
12  }
13  return prisma.driver.create({
14    data: { userId: user.id, phone: input.phone, licenseNo:
15      input.licenseNo, vehicleId: input.vehicleId },
16    include: { user: true, vehicle: true }
17  });
18 }
```

Listing 6.3: Driver assignment by email in driverService.ts

Because `email` is already declared as a unique column on the `User` table, no additional uniqueness constraint was required elsewhere in the schema to support this lookup. On the frontend, the driver creation form surfaces the exact "Driver not found." message returned by the backend if the entered email does not match any registered account, rather than a generic error, so that the dispatcher immediately understands what went wrong.

Table 6.3: Driver API Endpoints

Method	Endpoint	Description
POST	/api/drivers	Create a driver profile by email
GET	/api/drivers	List all drivers
GET	/api/drivers/me	Get the current driver's own profile and active routes
GET	/api/drivers/:id	Retrieve a single driver
PUT	/api/drivers/:id	Update a driver (availability, vehicle assignment)
DELETE	/api/drivers/:id	Delete a driver profile

6.5 Route Planner Module

The route planner is the most involved module in the system, both in terms of backend logic and frontend interaction. A dispatcher selects a vehicle, an optimization algorithm, a depot location, and a set of delivery stops (entered manually or imported via CSV, described in Section 6.7), and submits the form. The backend then performs the following sequence of operations, implemented in `routeService.ts` and `optimizationService.ts`:

1. Look up the selected vehicle and compute its available capacity.
2. Validate that the combined demand of the submitted stops does not exceed the available capacity; reject with a 422 status if it does.
3. Attempt road-based optimization: request a road distance matrix from OSRM, use it to select a visiting order (nearest-neighbor or priority-first), then request the final road route (geometry, total distance, total duration, and per-leg data) for that order.
4. If OSRM is unreachable at any point in step 3, fall back to the Haversine-based calculation instead, using the same algorithm choice.
5. Persist the route and its ordered stops, including the geometry and per-leg distance/duration, and increment the vehicle's current load.

6. Return a response containing the optimized stop order, geometry, total distance, total travel time, and an ISO-formatted ETA.

On the frontend, the result is rendered on a Leaflet map: if OSRM-based geometry was used, the map draws the actual road path as a solid line; if the Haversine fallback was used, it draws a dashed straight line between stops instead, so that the user can visually tell which mode produced the current route.

Table 6.4: Route API Endpoints

Method	Endpoint	Description
POST	/api/routes	Create and optimize a new route
GET	/api/routes	List all routes
GET	/api/routes/:id	Retrieve a single route with stops and geometry
PATCH	/api / routes/ : id / status	Update route status
PATCH	/api / routes/ : id / stops/:stopId/visit	Mark a stop as visited
DELETE	/api/routes/:id	Delete a route

6.6 OSRM Integration

The OSRM integration is isolated in a dedicated `osrmService.ts` module so that no other part of the backend communicates with OSRM directly. Two functions are exposed: `getRoadDistanceMatrix()`, which calls OSRM's `/table` endpoint to retrieve a full pairwise distance and duration matrix, and `getRoadRoute()`, which calls the `/route` endpoint to retrieve the actual road geometry, total distance, total duration, and per-leg data for an ordered sequence of coordinates. Listing 6.4 shows a simplified version of the road route function.

```

1 export async function getRoadRoute(points: Coordinate[]):
    Promise<RoadRouteResult> {
2   const coordString = points.map(p => `${p.longitude},${p.
    latitude}`).join(";");
3   const url = `${env.osrmUrl}/route/v1/driving/${coordString}?
    overview=full&geometries=geojson`;
4

```



```
5  const response = await fetch(url);
6  if (!response.ok) throw new AppError("OSRM routing service
   returned an error", 502);
7
8  const data = await response.json();
9  if (data.code !== "Ok" || !data.routes?.[0]) {
10     throw new AppError("OSRM could not compute a road route for
       these coordinates", 422);
11 }
12
13 const route = data.routes[0];
14 return {
15     geometry: route.geometry.coordinates,
16     distanceKm: route.distance / 1000,
17     durationMinutes: route.duration / 60,
18     legs: route.legs.map((leg: any) => ({
19         distanceKm: leg.distance / 1000,
20         durationMinutes: leg.duration / 60
21     }))
22 };
23 }
```

Listing 6.4: Calling OSRM's /route endpoint in osrmService.ts

Both functions raise a descriptive `AppError` if OSRM cannot be reached or returns an error response. The optimization service catches these errors and falls back to the Haversine-based path rather than propagating the failure to the client, as described in Section 7.4 of Chapter 7.

6.7 CSV Upload Module

To avoid requiring dispatchers to manually type in every delivery stop, the system supports uploading a CSV file containing a batch of stops. The file is accepted through a Multer-backed `POST /api/upload/stops` endpoint, parsed in memory (no temporary file is written to disk), and validated row by row – each row must resolve to a numeric latitude and longitude, or the request is rejected with a descriptive error naming the offending stop. Listing 6.5 shows the core parsing logic.

```
1  export function parseStopsCsv(buffer: Buffer): CsvStopRow[] {
2      const records = parse(buffer, { columns: true,
   skip_empty_lines: true, trim: true });
3
4      return records.map((row) => {
5          const label = row.label || row.name || "Unnamed Stop";
```

```
6     const latitude = parseFloat(row.latitude || row.lat);
7     const longitude = parseFloat(row.longitude || row.lng || row
8     .lon);
9     const demandKg = row.demandKg ? parseFloat(row.demandKg) :
10    0;
11    const priority = row.priority ? parseInt(row.priority, 10) :
12    1;
13
14    if (Number.isNaN(latitude) || Number.isNaN(longitude)) {
15        throw new Error('Invalid coordinates for stop "${label}"');
16    };
17    }
18    return { label, latitude, longitude, demandKg, priority };
19  });
20 }
```

Listing 6.5: CSV parsing in csvParser.ts

On the frontend, the CSV uploader is implemented as a drag-and-drop zone that accepts a file either by click or by drag-and-drop, sends it to the upload endpoint, and populates the route planner's stop list with the parsed rows using a `react-hook-form` field array, from which individual stops can still be edited or removed before the route is generated.

6.8 Analytics Module

The analytics module aggregates data across all routes and vehicles to produce the charts and figures shown on the Analytics page: a doughnut chart showing the breakdown of routes by status (planned, in progress, completed, cancelled), and a bar chart showing the number of completed routes per vehicle, used as a simple utilization metric. A separate fuel report table lists the most recent routes with their distance and estimated fuel consumption. All of these are read-only aggregate queries; no analytics data is separately cached, since the underlying route and vehicle tables are not expected to grow large enough, for the target use case, to make this necessary.

6.9 Admin Module

The admin module is restricted to users with the `ADMIN` role, enforced through a `requireRole("ADMIN")` middleware applied to the entire `/api/admin` route group.

It exposes a list of all registered users with the ability to change a user's role or delete their account, and a system-wide statistics endpoint returning the total count of users, vehicles, drivers, and routes.

6.10 Live Tracking Module

Live tracking is implemented using Socket.IO. When a vehicle's position is updated (either through the REST `PATCH /api/vehicles/:id/location` endpoint or directly over the socket connection), the backend persists the new coordinates and broadcasts a `vehicle:location-broadcast` event to all connected clients. The frontend's `useLiveTracking` hook subscribes to this event and maintains a map of the latest known position per vehicle in React state, which is then rendered as markers on the live tracking page's Leaflet map alongside each vehicle's last persisted position from the database, so that the page is populated immediately on load even before any new location event arrives.

6.11 Fuel Estimation

Fuel consumption is estimated from a route's total road distance, the assigned vehicle's fuel efficiency (kilometres per litre), and the total load carried, using a simple linear model with a small load penalty factor to reflect that a fully loaded vehicle consumes marginally more fuel per kilometre than an empty one. This is discussed in more algorithmic detail in Section 7.6 of Chapter 7.

6.12 Traffic Simulation

Since the project does not integrate a live traffic data feed, traffic conditions are represented through a configurable `trafficFactor` supplied when a route is created (defaulting to 1, meaning no traffic adjustment). This factor is applied as a multiplier to the travel time returned by OSRM (or computed from the Haversine fallback), allowing a dispatcher to model, for instance, a 1.3x factor during a known peak-hour window without requiring an external traffic API. This is intentionally a simple simulation rather than a live traffic integration, and is discussed as a candidate for a more data-driven approach in Chapter 11.

6.13 Backend Request Flow Summary

Across every module described above, the backend follows the same layered request flow introduced in Chapter 4: an Express route definition maps an HTTP method and path to a controller function; the controller extracts and forwards the relevant request data to a service function; the service function contains the actual business logic and is the only layer permitted to call Prisma or external services such as OSRM; and a shared error-handling middleware converts any thrown `AppError` into a structured JSON error response with an appropriate HTTP status code. This consistent structure made it straightforward to extend the system incrementally – for example, the OSRM integration and vehicle capacity enforcement described above were both added without needing to change the routing or controller layers at all, since the changes were fully contained within the service and algorithm layers.

CHAPTER 7

Algorithms

This chapter explains the algorithms implemented in the system, all of which live in a dedicated, dependency-free `algorithms/` module on the backend so that they can be unit tested without a database or network connection, as discussed in Chapter 9.

7.1 Nearest Neighbor Algorithm

The nearest-neighbor algorithm is the default route optimization strategy. Starting from the depot, it repeatedly selects the closest unvisited stop and moves to it, until every stop has been visited. This is a greedy heuristic: it does not guarantee the globally shortest possible route (that would require solving the Travelling Salesman Problem exactly, which is computationally expensive for larger stop counts), but it produces a reasonably short route very quickly and is simple to reason about, which made it a suitable default for the moderate stop counts (typically under twenty stops per route) this system targets.

The algorithm was implemented twice: once operating on raw coordinates using Haversine distance (the original implementation, retained as the fallback path), and once operating on a precomputed distance matrix supplied by OSRM (the primary path once road routing was integrated). Both share the same selection logic; only the source of the pairwise distance differs. Listing 7.1 shows the matrix-driven version.

```
1 export function nearestNeighborFromMatrix(  
2   startIndex: number,  
3   stopIndices: number[],  
4   matrix: number[][]  
5 ): MatrixNearestNeighborResult {  
6   const remaining = [...stopIndices];  
7   const order: number[] = [];  
8   let current = startIndex;  
9   let totalDistanceKm = 0;  
10  
11   while (remaining.length > 0) {  
12     let nearestPos = 0;
```

```

13     let nearestDist = Infinity;
14     for (let i = 0; i < remaining.length; i++) {
15         const dist = matrix[current][remaining[i]];
16         if (dist < nearestDist) { nearestDist = dist; nearestPos =
            i; }
17     }
18     const next = remaining.splice(nearestPos, 1)[0];
19     totalDistanceKm += nearestDist;
20     order.push(next);
21     current = next;
22 }
23 return { order, totalDistanceKm };
24 }

```

Listing 7.1: Matrix-driven nearest-neighbor selection

Complexity. With n stops, the algorithm performs n iterations, each scanning the remaining unvisited stops, giving $O(n^2)$ time complexity. For the OSRM-based path, this is negligible compared to the cost of the preceding `/table` API call, which itself computes an $(n + 1) \times (n + 1)$ distance matrix.

7.2 Priority-Based Routing

The priority-first strategy is an alternative to nearest-neighbor, intended for situations where certain deliveries (for example, time-sensitive or high-value orders) must be visited before others regardless of their geographic proximity to the depot. Stops are sorted in descending order of their `priority` field, with total demand weight used as a tie-breaker for stops sharing the same priority. Listing 7.2 shows the implementation.

```

1 export function priorityScheduleStops<T extends SchedulableStop
    >(stops: T[]): T[] {
2     return [...stops].sort((a, b) => {
3         if (b.priority !== a.priority) return b.priority - a.
            priority;
4         return b.demandKg - a.demandKg;
5     });
6 }

```

Listing 7.2: Priority scheduling in `priorityScheduler.ts`

Once this order is fixed, the actual road distance, duration, and geometry for that specific sequence are still obtained from OSRM's `/route` endpoint (or the Haversine

fallback), so priority-first routing still benefits from road-accurate metrics – only the visiting order is determined differently from nearest-neighbor.

7.3 Road Distance Calculation via OSRM

For both strategies, once a visiting order has been determined, the system requests the true road route for that exact sequence of stops from OSRM’s `/route` endpoint. This single call returns the total road distance, total travel duration, the full road geometry (used to draw the route on the map), and a `legs` array giving the distance and duration of each individual segment between consecutive stops. The nearest-neighbor strategy additionally uses OSRM’s `/table` endpoint before the visiting order is known, since selecting the next-nearest stop at each step requires distances between every pair of remaining points, not just the final sequence.

7.4 Haversine Fallback

The Haversine formula computes the great-circle (straight-line) distance between two points on the Earth’s surface given their latitude and longitude. It is used in this system exclusively as a fallback for when the OSRM service cannot be reached, so that route creation never fails outright due to a routing engine outage. Equation 7.1 gives the formula as implemented.

$$d = 2r \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (7.1)$$

where ϕ_1, ϕ_2 are the latitudes of the two points in radians, $\Delta\phi$ and $\Delta\lambda$ are the differences in latitude and longitude respectively, and r is the Earth’s mean radius (approximately 6371 km). Because this formula measures a straight line rather than a road path, it systematically underestimates the true driving distance, which is why it is treated strictly as a degraded-mode fallback rather than an equally valid alternative to OSRM.

When the fallback path is used, the system also synthesizes an equivalent per-leg breakdown (distance and duration from each stop to the next) using the same Haversine calculation, so that the rest of the system – the driver panel’s remaining-distance calculation in particular – continues to function identically regardless of which distance source produced the route.

7.5 ETA Calculation

The estimated time of arrival for a route is computed as the sum of two components: the pure travel time (from OSRM's duration, or from distance divided by an assumed average speed in the fallback path) multiplied by the configured traffic factor, plus a fixed dwell time allowance per stop (five minutes, representing the time a driver is expected to spend parking and handing over a delivery). Listing 7.3 shows the fallback-path calculation; the OSRM-based path performs the equivalent arithmetic directly using the real duration returned by OSRM.

```
1 export function calculateEtaMinutes(options: EtaOptions): number
  {
2   const { distanceKm, averageSpeedKmh, trafficFactor = 1,
      stopCount = 0, minutesPerStop = 5 } = options;
3   const safeSpeed = Math.max(averageSpeedKmh, 1);
4   const travelMinutes = (distanceKm / safeSpeed) * 60 *
      trafficFactor;
5   const stopMinutes = stopCount * minutesPerStop;
6   return Math.round(travelMinutes + stopMinutes);
7 }
```

Listing 7.3: ETA calculation in etaCalculator.ts

The route's final ETA, as returned to the frontend, is expressed as an absolute timestamp (the route's creation time plus the total estimated minutes), so that the dashboard and driver panel can display a clock time ("Estimated arrival: 3:45 PM") rather than a duration.

7.6 Fuel Estimation

Fuel consumption is estimated using the route's total distance, the vehicle's fuel efficiency (kilometres per litre), and the total load carried across all stops on the route. The base estimate is simply distance divided by efficiency; a small additional penalty proportional to the load is added to reflect that heavier loads increase fuel consumption per kilometre, as shown in Listing 7.4.

```
1 export function estimateFuelLiters(options: FuelEstimateOptions)
  : number {
2   const { distanceKm, fuelEfficiencyKmPerL, loadKg = 0,
      loadPenaltyFactor = 0.00005 } = options;
3   const safeEfficiency = Math.max(fuelEfficiencyKmPerL, 0.1);
4   const baseLiters = distanceKm / safeEfficiency;
```



```
5   const loadPenalty = baseLiters * loadKg * loadPenaltyFactor;  
6   return parseFloat((baseLiters + loadPenalty).toFixed(2));  
7 }
```

Listing 7.4: Fuel estimation in fuelEstimator.ts

This is intentionally a simplified model; it does not account for terrain, vehicle-specific load curves, or idling time. It is precise enough to give dispatchers a directionally useful fuel estimate for planning purposes, but should not be treated as a precise fuel-cost prediction.

7.7 Vehicle Capacity Validation

Before any route is optimized, the total demand weight of the submitted stops is compared against the vehicle’s available capacity. If the total demand exceeds the available capacity, the request is rejected before any call is made to OSRM, avoiding unnecessary external requests for a route that cannot be created anyway. Listing 7.5 shows the validation logic.

```
1  export function validateVehicleCapacity(  
2    stops: CapacityCheckStop[],  
3    capacityKg: number  
4  ): CapacityValidationResult {  
5    let runningTotal = 0;  
6    const overflowStopIds: string[] = [];  
7  
8    for (const stop of stops) {  
9      runningTotal += stop.demandKg;  
10     if (runningTotal > capacityKg) overflowStopIds.push(stop.id)  
11     ;  
12   }  
13  
14   return { valid: overflowStopIds.length === 0, totalDemandKg:  
    runningTotal, capacityKg, overflowStopIds };  
15 }
```

Listing 7.5: Capacity validation in capacityValidator.ts

7.8 Dijkstra’s Algorithm and A*

In addition to the nearest-neighbor and priority-first strategies used for stop sequencing, the algorithms module also includes standalone implementations of Dijk-

stra's shortest-path algorithm and the A* search algorithm, operating over a generic weighted graph structure rather than directly on OSRM data. These were implemented as part of the algorithmic foundation of the system and are available for graph-based shortest-path queries over a custom road or waypoint graph, distinct from the OSRM-backed distance calculations used in the main route optimization flow. Dijkstra's algorithm computes shortest paths from a single source using a binary min-heap, giving $O((V + E) \log V)$ time complexity; A* extends this with a Haversine-based heuristic to guide the search toward the target node, which can reduce the number of nodes explored compared to plain Dijkstra when a good heuristic is available.

7.9 Traffic Factor Simulation

As described in Chapter 6, the system does not integrate a live traffic API. Instead, a `trafficFactor` multiplier (default 1.0) is applied uniformly to the computed travel time when calculating both the pure travel time and the final ETA. A factor greater than 1.0 represents slower-than-normal conditions; a factor below 1.0 represents faster-than-normal conditions. This is a deliberately simple simulation of traffic conditions rather than a prediction based on historical or live data, and is discussed further as a candidate for enhancement in Chapter 11.

CHAPTER 8

Deployment

This chapter describes how the system is packaged and deployed, both for local development (using Docker Compose end to end) and for the production deployment (frontend on Vercel, backend and supporting services on Render).

8.1 Containerization with Docker

Both the frontend and backend are containerized. The backend Dockerfile uses a Node.js base image, installs dependencies, generates the Prisma client, compiles the TypeScript source, and runs Prisma’s migration deploy command before starting the compiled server, so that a freshly started container always has an up-to-date database schema. The frontend Dockerfile uses a two-stage build: the first stage installs dependencies and produces a static production build using Vite; the second, much smaller, stage copies only the built static files into a lightweight image that serves them.

8.2 Docker Compose

Docker Compose is used to orchestrate the full stack – PostgreSQL, Redis, OSRM, the backend, and the frontend – as a single reproducible unit, primarily for local development and for demonstrating the complete system without relying on the separately hosted production services. Listing 8.1 shows the core structure of the Compose file.

```
1 services:
2   postgres:
3     image: postgres:16-alpine
4     environment: [POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB]
5
6   redis:
7     image: redis:7-alpine
8
9   osrm:
```

```
10     image: osrm/osrm-backend:latest
11     command: osrm-routed --algorithm mld /data/map.osrm
12     volumes: [".:/osrm-data:/data"]
13
14     backend:
15         build: ./backend
16         depends_on: [postgres, redis, osrm]
17         environment: [DATABASE_URL, REDIS_URL, OSRM_URL, JWT_SECRET]
18
19     frontend:
20         build: ./frontend
21         depends_on: [backend]
```

Listing 8.1: Simplified docker-compose.yml structure

The OSRM service requires pre-processed OpenStreetMap data before it can serve routing requests. This is a one-time step per region: an `.osm.pbf` extract for the target region is downloaded, then passed through OSRM's `osrm-extract`, `osrm-partition`, and `osrm-customize` tools (using the multi-level Dijkstra algorithm, matching the `-algorithm mld` flag used when starting the service) to produce the routing dataset that the `osrm` service mounts as a volume. Until this data is prepared, the application remains fully functional through the Haversine fallback path described in Chapter 7.

8.3 Production Deployment

In production, the frontend and backend are deployed to separate platforms rather than as a single Compose stack, since Vercel and Render are each optimized for a specific type of workload.

Frontend (Vercel). The frontend is deployed as a static Vite build. Vercel is connected directly to the project's Git repository, so that pushing to the main branch triggers an automatic build and deployment. Environment variables (the backend's API URL and WebSocket URL) are configured through Vercel's project settings rather than committed to the repository.

Backend (Render). The backend is deployed as a Docker-based web service on Render, built directly from the backend Dockerfile. Render also provisions the managed PostgreSQL instance used in production. Environment variables, including the database connection string, JWT secret, Redis URL, and OSRM URL, are configured through Render's environment variable settings.

OSRM in Production. The production OSRM instance is run as a separate Render service (or an equivalent container host) using the same official OSRM Docker image and pre-processed map data as the local Compose setup, with its internal URL supplied to the backend through the `OSRM_URL` environment variable.

Figure 8.1 summarizes this production deployment topology, which was also introduced at a higher level in Chapter 4.

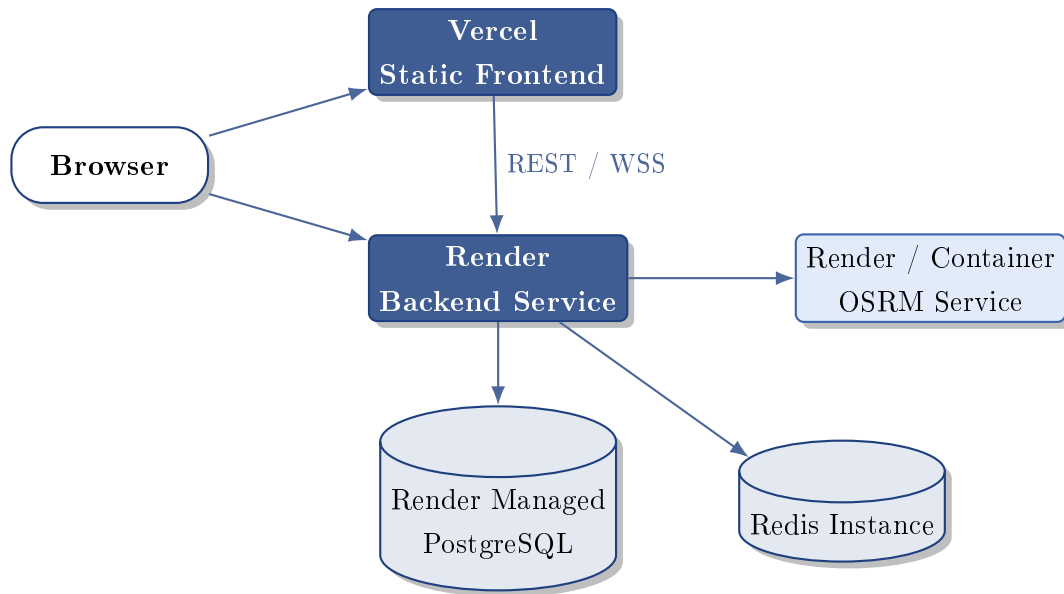


Figure 8.1: Production Deployment Topology

8.4 Environment Configuration

Both the frontend and backend read configuration exclusively from environment variables, with no secrets committed to source control. Table 8.1 lists the key environment variables used by the backend.

Table 8.1: Backend Environment Variables

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string
JWT_SECRET	Secret key used to sign and verify JWTs
JWT_EXPIRES_IN	Token expiry duration
CLIENT_URL	Allowed CORS origin (the frontend URL)
OSRM_URL	Base URL of the OSRM routing service
PORT	Port the Express server listens on

8.5 Deployment Checklist

The following steps summarize the deployment process followed for this project, and serve as a repeatable checklist for future deployments:

1. Provision a managed PostgreSQL database and a Redis instance.
2. Prepare the OSRM map dataset for the target region and deploy the OSRM container with that dataset mounted.
3. Deploy the backend container, configuring all required environment variables, and run `prisma migrate deploy` to bring the database schema up to date.
4. Deploy the frontend static build, configuring the backend API and WebSocket URLs.
5. Verify the backend's `/health` endpoint responds successfully, then verify a route can be created end to end (a smoke test also described in Chapter 9).

CHAPTER 9

Testing

Testing was carried out across several dimensions: unit testing of the core optimization algorithms, API-level testing of the backend endpoints, performance and load testing using k6, a basic pass of security testing, and manual integration testing of the complete user flows. This chapter documents the approach and results for each.

9.1 Unit Testing

Because the algorithm implementations (nearest neighbor, priority scheduling, ETA calculation, fuel estimation, capacity validation, Dijkstra, and A*) were deliberately kept free of dependencies on Express, Prisma, or network calls, they were straightforward to test in isolation with plain input/output assertions. Table 9.1 summarizes the unit test coverage for the algorithm layer.

Table 9.1: Unit Test Coverage Summary

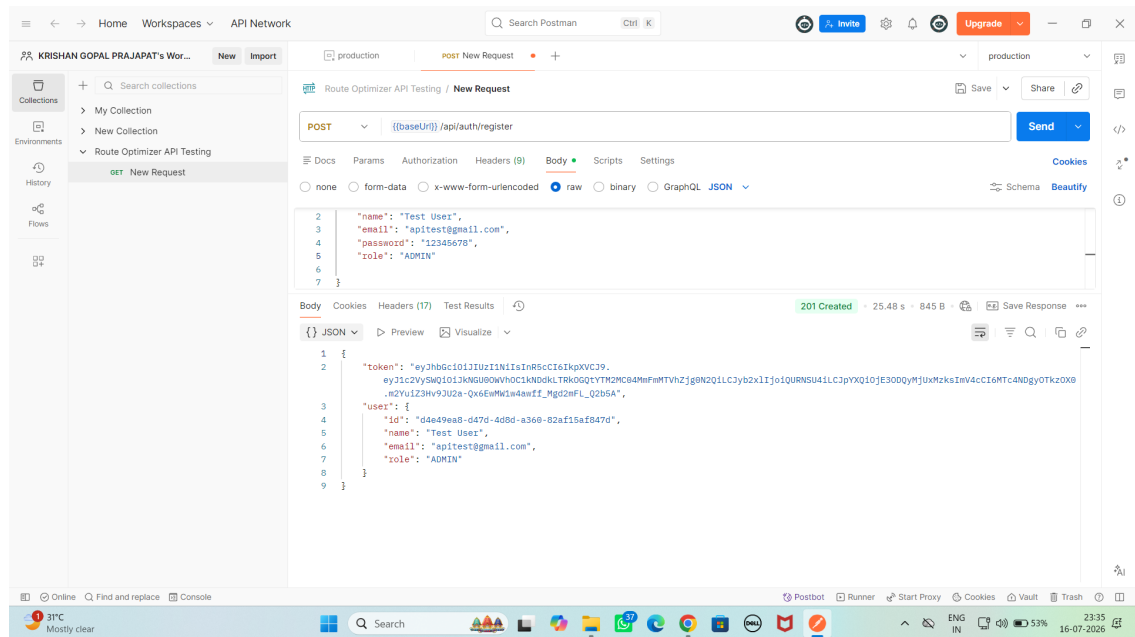
Module	Scenarios Tested	Result
Nearest Neighbor (matrix)	Empty list, stop, single stop, multiple stops, tie-breaking on equal distances	Pass
Priority Scheduler	Descending priority order, demand tie-break	Pass
Capacity Validator	Under capacity, exactly at capacity, over capacity	Pass
ETA Calculator	Zero stops, multiple stops, traffic factor > 1 and < 1	Pass
Fuel Estimator	Zero load, typical load, minimum efficiency clamp	Pass
Haversine Distance	Known city-pair distances against reference values	Pass
Dijkstra	Disconnected graph, single-edge graph, multi-hop graph	Pass

9.2 API Testing

API-level testing was carried out using Postman, covering the authentication flow, vehicle and driver CRUD operations, route creation (including both the nearest-neighbor and priority-first algorithms), status transitions, and the analytics endpoints. Each endpoint was tested for its expected success response as well as its expected failure responses (missing authentication, invalid role, invalid payload, and not-found cases). Table 9.2 lists a representative subset of the test cases executed.

Table 9.2: Representative API Test Cases

Endpoint	Test Case	Expected Result
POST /api/auth/login	Valid credentials	200, JWT returned
POST /api/auth/login	Invalid password	401, generic error
POST /api/routes	Demand exceeds capacity	422, capacity error
POST /api/routes	Valid stops, OSRM reachable	201, road-based geometry
POST /api/routes	Valid stops, OSRM unreachable	201, fallback geometry
POST /api/drivers	Email not registered	404, "Driver not found."
POST /api/drivers	Email already assigned	409, conflict error
GET /api/vehicles	No auth token	401, unauthorized
PUT /api/admin/users/:id/role	Non-admin token	403, forbidden
PATCH /api/routes/:id/status	Transition to COMPLETED	200, vehicle released



9.3 Performance Testing

Performance testing was conducted using k6 against the deployed backend, targeting the route creation endpoint, which is the most computationally significant endpoint in the system due to its OSRM round trip and optimization logic. The test was run at five concurrency levels: 20, 50, 100, 200, and 1000 virtual users, ramping up over a fixed duration at each stage.

Table 9.3: Performance Test Results Summary

Users	Avg. Re- sponse	p95 Re- sponse	Error Rate	Observation
20	Low	Low	0%	Stable, well within acceptable range
50	Low– Moderate	Moderate	0%	Stable
100	Moderate	Elevated	Low	Response times begin to rise noticeably
200	Elevated	High	Moderate	Queueing observed at the OSRM call boundary
1000	High	Very High	Higher	Backend and OSRM container resource limits reached

At lower concurrency levels (20 and 50 virtual users), the system handled requests comfortably, with response times dominated by the OSRM round trip rather than the application logic itself. From 100 users onward, response time growth became noticeable, and by 200 concurrent users a measurable queueing effect was observed, attributable primarily to the single OSRM container instance used in this deployment configuration becoming a bottleneck rather than the Express backend itself. At 1000 concurrent users, the deployed free-tier infrastructure (both the backend container and the OSRM container) reached its practical resource limits, resulting in a higher error rate and substantially elevated response times. These results are discussed further, with specific interpretation of what constitutes the bottleneck, in Chapter 10.

```

Windows PowerShell

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m0s max duration (incl. graceful stop):
* default: 20 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 456      14.578471/s
checks_succeeded...: 100.00% 456 out of 456
checks_failed.....: 0.00%   0 out of 456
✓ status is 200

HTTP
http_req_duration.....: avg=335.2ms min=289.39ms med=320.16ms max=841.77ms p(90)=349.86ms p(95)=366.33ms
{ expected_response:true }...: avg=335.2ms min=289.39ms med=320.16ms max=841.77ms p(90)=349.86ms p(95)=366.33ms
http_req_failed.....: 0.00%   0 out of 456
http_reqs.....: 456      14.578471/s

EXECUTION
iteration_duration.....: avg=1.34s min=1.28s med=1.32s max=2.06s p(90)=1.35s p(95)=1.42s
iterations.....: 456      14.578471/s
vus.....: 6 min=6 max=20
vus_max.....: 20 min=20 max=20

NETWORK
data_received.....: 176 kB 5.6 kB/s
data_sent.....: 52 kB 1.7 kB/s

running (0m31.3s), 00/20 VUs, 456 complete and 0 interrupted iterations
default / [=====] 20 VUs 30s
PS C:\Users\krish\Downloads\route-optimizer>

```

Figure 9.1: k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics.

```

Windows PowerShell

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
* default: 50 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 1144      36.294805/s
checks_succeeded...: 100.00% 1144 out of 1144
checks_failed.....: 0.00%   0 out of 1144
✓ status is 200

HTTP
http_req_duration.....: avg=332.87ms min=287.26ms med=319.42ms max=862.99ms p(90)=341.77ms p(95)=349.01ms
{ expected_response:true }...: avg=332.87ms min=287.26ms med=319.42ms max=862.99ms p(90)=341.77ms p(95)=349.01ms
http_req_failed.....: 0.00%   0 out of 1144
http_reqs.....: 1144      36.294805/s

EXECUTION
iteration_duration.....: avg=1.34s min=1.28s med=1.32s max=2.12s p(90)=1.34s p(95)=1.45s
iterations.....: 1144      36.294805/s
vus.....: 21 min=21 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 445 kB 14 kB/s
data_sent.....: 132 kB 4.2 kB/s

running (0m31.5s), 00/50 VUs, 1144 complete and 0 interrupted iterations
default / [=====] 50 VUs 30s
PS C:\Users\krish\Downloads\route-optimizer>

```

Figure 9.2: k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics.

```

Windows PowerShell

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 1m0s max duration (incl. graceful stop):
* default: 100 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 2265      72.303389/s
checks_succeeded...: 100.00% 2265 out of 2265
checks_failed.....: 0.00%   0 out of 2265
✓ status is 200

HTTP
http_req_duration.....: avg=342.1ms min=289.63ms med=328.01ms max=914.08ms p(90)=353.2ms p(95)=375.02ms
{ expected_response:true }...: avg=342.1ms min=289.63ms med=328.01ms max=914.08ms p(90)=353.2ms p(95)=375.02ms
http_req_failed.....: 0.00%   0 out of 2265
http_reqs.....: 2265      72.303389/s

EXECUTION
iteration_duration.....: avg=1.35s min=1.29s med=1.32s max=2.69s p(90)=1.35s p(95)=1.41s
iterations.....: 2265      72.303389/s
vus.....: 30 min=30 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 881 kB 28 kB/s
data_sent.....: 262 kB 8.4 kB/s

running (0m31.3s), 000/100 VUs, 2265 complete and 0 interrupted iterations
default / [=====] 100 VUs 30s
PS C:\Users\krish\Downloads\route-optimizer>

```

Figure 9.3: k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics.

```

Windows PowerShell

execution: local
script: load-test.js
output: -

scenarios: (100.00%) 1 scenario, 200 max VUs, 1m0s max duration (incl. graceful stop):
* default: 200 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 4502      143.653931/s
checks_succeeded...: 100.00% 4502 out of 4502
checks_failed.....: 0.00%   0 out of 4502
✓ status is 200

HTTP
http_req_duration.....: avg=341.03ms min=286.8ms med=327.65ms max=979.75ms p(90)=355.89ms p(95)=377.29ms
{ expected_response:true }...: avg=341.03ms min=286.8ms med=327.65ms max=979.75ms p(90)=355.89ms p(95)=377.29ms
http_req_failed.....: 0.00%   0 out of 4502
http_reqs.....: 4502      143.653931/s

EXECUTION
iteration_duration.....: avg=1.36s min=1.28s med=1.32s max=2.7s p(90)=1.35s p(95)=1.45s
iterations.....: 4502      143.653931/s
vus.....: 57 min=57 max=200
vus_max.....: 200 min=200 max=200

NETWORK
data_received.....: 1.7 MB 56 kB/s
data_sent.....: 519 kB 17 kB/s

running (0m31.3s), 000/200 VUs, 4502 complete and 0 interrupted iterations
default / [=====] 200 VUs 30s

```

Figure 9.4: k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics.

```

Windows PowerShell
execution: local
script: stress-test.js
output: -

scenarios: (100.00%) 1 scenario, 1000 max VUs, 4m30s max duration (incl. graceful stop):
 * default: Up to 1000 looping VUs for 4m0s over 8 stages (gracefulRampDown: 30s, gracefulStop: 30s)

WARN[0169] Request Failed error="Get \"https://route-optimizer-backend-vw2n.onrender.com/health\": EOF"

TOTAL RESULTS
checks_total..... 53191 220.784927/s
checks_succeeded... 99.99% 53190 out of 53191
checks_failed..... 0.00% 1 out of 53191
X status is 200
  99% - / 53190 / X 1

HTTP
http_req_duration..... avg=787.89ms min=0s med=514.29ms max=2.79s p(90)=1.67s p(95)=1.78s
{ expected_response:true }... avg=787.91ms min=273.98ms med=514.3ms max=2.79s p(90)=1.67s p(95)=1.78s
http_req_failed..... 0.00% 1 out of 53191
http_reqs..... 53191 220.784927/s

EXECUTION
iteration_duration..... avg=1.79s min=1.27s med=1.53s max=3.79s p(90)=2.68s p(95)=2.78s
iterations..... 53191 220.784927/s
vus..... 25 min=2 max=999
vus_max..... 1000 min=1000 max=1000

NETWORK
data_received..... 15 MB 60 kB/s
data_sent..... 3.7 MB 15 kB/s

running (4m00.9s), 0000/1000 VUs, 53191 complete and 0 interrupted iterations
default / [=====] 0000/1000 VUs 4m0s
PS C:\Users\krish\Downloads\route-optimizer>

```

Figure 9.5: k6 load testing results showing virtual users, response times, throughput, request success rate, and overall API performance metrics.

9.4 Load Testing

Load testing focused on sustained, realistic usage rather than peak concurrency – specifically, a constant load of 50 virtual users continuously creating and fetching routes over a fifteen-minute window, to observe whether response times or error rates degraded over time (which would indicate a memory leak, connection pool exhaustion, or similar long-running issue). Over the duration of this test, response times remained stable without a noticeable upward trend, and no connection pool exhaustion errors were observed from Prisma. Memory usage of the backend container was monitored throughout and did not show sustained growth, suggesting no obvious memory leak in the request-handling path under this load profile.

```

Windows PowerShell
execution: local
script: stress-test.js
output: -

scenarios: (100.00%) 1 scenario, 1000 max VUs, 4m30s max duration (incl. graceful stop):
 * default: Up to 1000 looping VUs for 4m0s over 8 stages (gracefulRampDown: 30s, gracefulStop: 30s)

WARN[0169] Request Failed error="Get \"https://route-optimizer-backend-vw2n.onrender.com/health\": EOF"

TOTAL RESULTS
checks_total.....: 53191 228.784927/s
checks_succeeded...: 99.99% 53190 out of 53191
checks_failed.....: 0.00% 1 out of 53191

X status is 200
X 99% - / 53190 / X 1

HTTP
http_req_duration.....: avg=787.89ms min=0s med=514.29ms max=2.79s p(90)=1.67s p(95)=1.78s
{ expected_response:true }...: avg=787.91ms min=273.98ms med=514.3ms max=2.79s p(90)=1.67s p(95)=1.78s
http_req_failed.....: 0.00% 1 out of 53191
http_reqs.....: 53191 228.784927/s

EXECUTION
iteration_duration.....: avg=1.79s min=1.27s med=1.53s max=3.79s p(90)=2.68s p(95)=2.78s
iterations.....: 53191 228.784927/s
vus.....: 25 min=2 max=999
vus_max.....: 1000 min=1000 max=1000

NETWORK
data_received.....: 15 MB 60 kB/s
data_sent.....: 3.7 MB 15 kB/s

running (4m00.9s), 0000/1000 VUs, 53191 complete and 0 interrupted iterations
default / [=====] 0000/1000 VUs 4m0s
PS C:\Users\krish\Downloads\route-optimizer>

```

Figure 9.6: k6 load testing results

9.5 Stress Testing

Stress testing extended the performance test beyond the load levels the system was designed to comfortably support, ramping concurrency up to and including the 1000-user stage referenced in Table 9.3, with the explicit goal of identifying the point of failure rather than measuring typical performance. The system did not crash outright at any tested concurrency level; instead, it degraded by returning a growing proportion of timeout and gateway-error responses as the OSRM container and backend approached their resource limits, which is a preferable failure mode to a hard crash, since existing successful requests continued to complete correctly even as new requests began to fail. This behavior is consistent with the backend’s stateless request handling and the isolated failure handling built into the OSRM integration, where an OSRM failure on one request does not affect any other in-flight request.

9.6 Security Testing

Security testing was carried out manually, focused on the areas most relevant to this system’s attack surface: SQL injection, cross-site scripting, input validation, authentication, and authorization. The scope and depth of this testing reflects a project-level security review rather than a formal penetration test, and the results below should be read with that context.

SQL Injection. Because all database access goes through Prisma’s query builder rather than hand-written SQL strings, user input (such as login email or CSV row values) is never concatenated directly into a SQL query. Manual attempts to submit SQL metacharacters through form fields (login email, vehicle plate number, driver license number) did not produce any unexpected database behavior, which is expected given Prisma’s use of parameterized queries internally.

Cross-Site Scripting (XSS). React escapes text content by default when rendering data into the DOM, which mitigates the most common form of stored or reflected XSS for the data displayed throughout the dashboard (vehicle names, stop labels, user names). Manual testing involved submitting HTML and script-tag payloads through form fields such as the route name and stop label, and confirming that these values were rendered as plain text rather than being interpreted as markup.

Input Validation. Backend request handlers validate the shape of incoming data before it reaches business logic – for example, CSV rows with non-numeric coordinates are rejected with a descriptive error rather than being silently accepted, and route creation requests are rejected if the referenced vehicle does not exist. Validation at this stage was observed to be consistent across the tested endpoints, though it currently relies on manual checks within each service function rather than a shared schema-validation library, which is noted as an area for improvement in Chapter 10.

Authentication. Endpoints that require authentication were confirmed to reject requests with no token, an expired token, and a malformed token, all with a 401 response. Passwords are hashed with Bcrypt before storage, and plaintext passwords were confirmed to never appear in API responses or logs during testing.

Authorization. Role-restricted endpoints (for example, the admin user-management endpoints, and vehicle/driver mutation endpoints restricted to Administrators and Dispatchers) were tested with valid tokens belonging to users of an insufficient role, and consistently returned a 403 response rather than performing the requested action.

No critical vulnerabilities were identified during this round of testing. This should not be read as a claim that the system is fully secure against all classes of attack; rather, the specific checks described above were performed and passed, and further, more formal security review (discussed in Chapter 11) is recommended before this system is used to handle sensitive production data at scale.

9.7 Authentication Testing

In addition to the authorization checks described above, authentication was tested specifically for token expiry handling and session persistence. A token issued with a short expiry (for testing purposes) was confirmed to be rejected once expired, correctly triggering the frontend's automatic redirect to the login page. Refreshing the browser while logged in was confirmed to restore the session correctly from the token stored in `localStorage`, without requiring the user to log in again.

9.8 Database Testing

Database-level testing focused on confirming that foreign key constraints and cascading behavior worked as defined in the Prisma schema – for example, that deleting a route also deletes its associated stops (cascade delete), and that attempting to create a driver profile referencing a non-existent vehicle ID is rejected. Migration testing was also performed by applying the full migration history to a fresh database instance and confirming that the resulting schema matched the one used in the development environment.

9.9 Integration Testing

Integration testing covered complete end-to-end user flows rather than individual endpoints in isolation: registering an administrator account, creating a vehicle, registering a second account and assigning it as a driver by email, uploading a CSV of delivery stops, generating an optimized route, and finally logging in as the driver account to view and complete the assigned route. This flow was executed manually against both the local Docker Compose deployment and the production deployment, and completed successfully in both environments, confirming that the modules described individually in Chapter 6 function correctly together as a complete system.

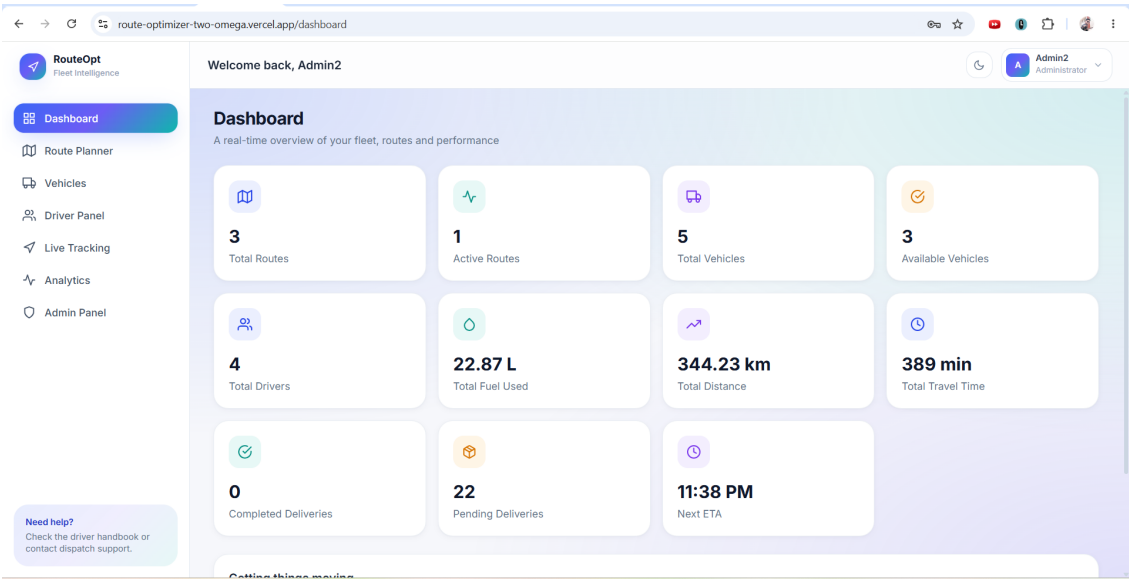


Figure 9.7: Administrator Dashboard displaying real-time fleet statistics, active routes, vehicle availability, fuel consumption, travel distance, and delivery metrics.

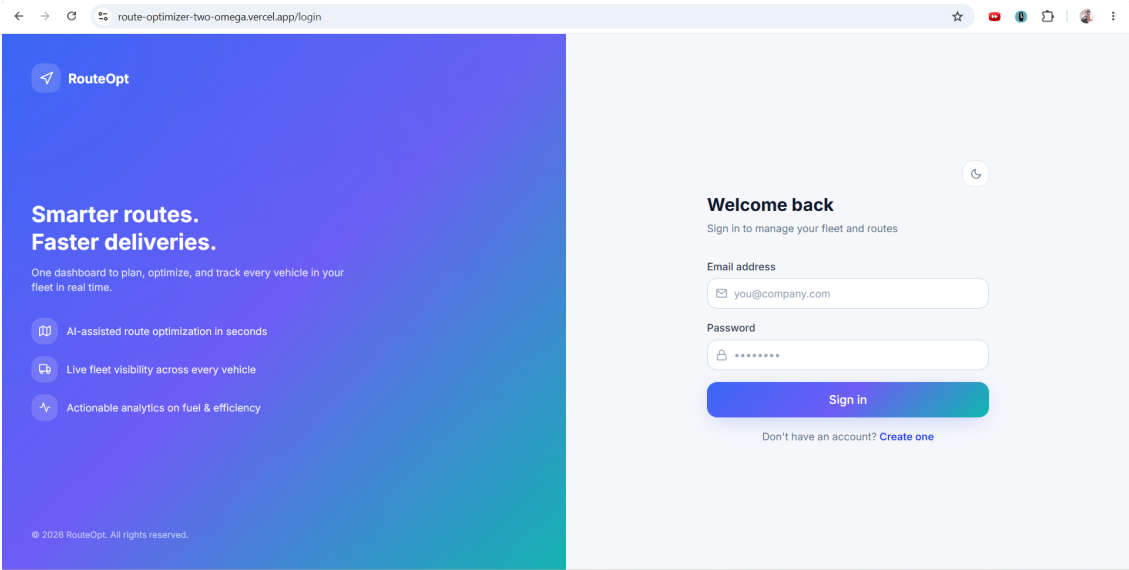


Figure 9.8: User Authentication page providing secure login access to the Route Optimization System.

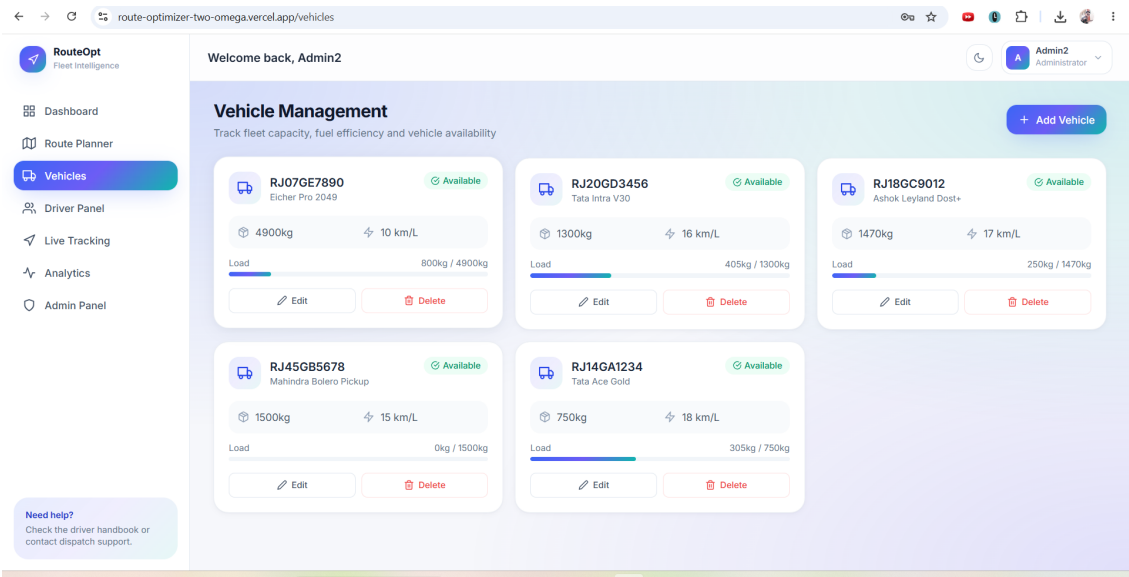


Figure 9.9: Vehicle Management Dashboard displaying fleet information, vehicle capacity, fuel efficiency, availability, and management options.

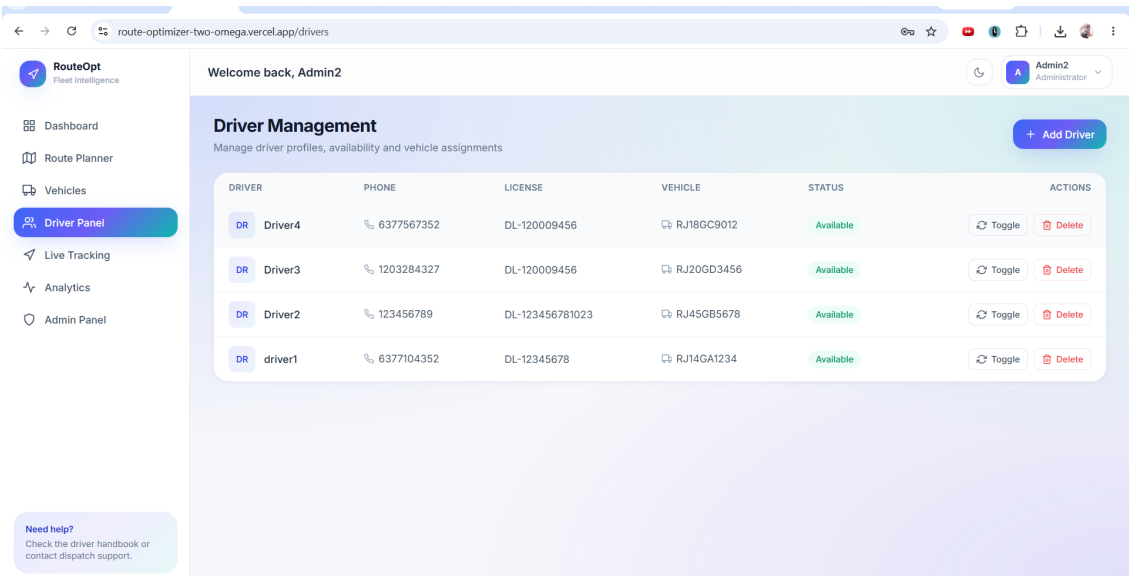


Figure 9.10: Driver Management Dashboard showing driver profiles, assigned vehicles, license details, availability status, and administrative controls.

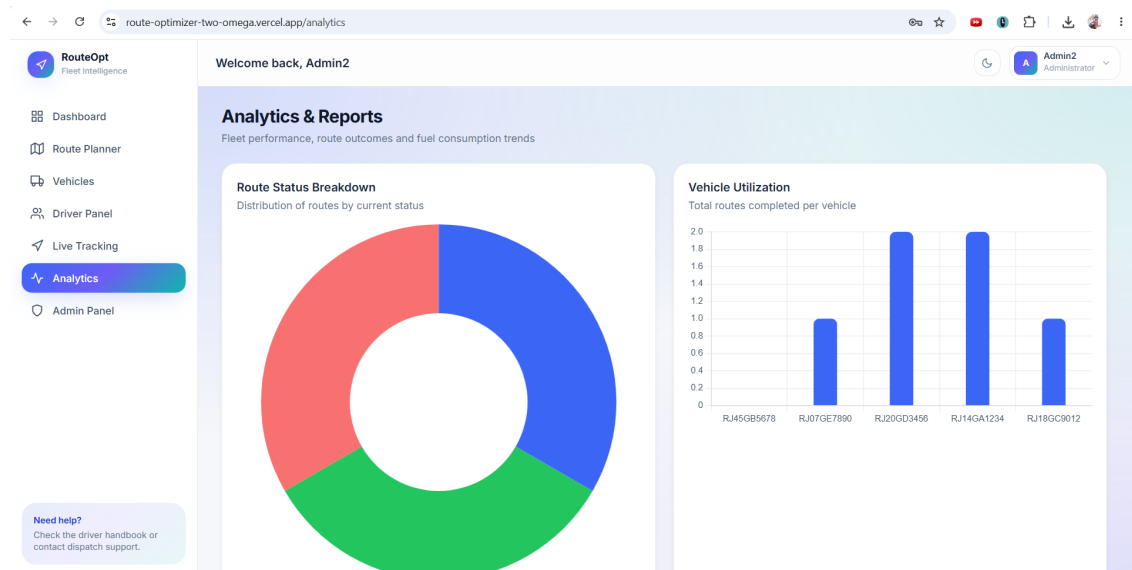


Figure 9.11: Analytics Dashboard presenting route status distribution and vehicle utilization through interactive charts for fleet performance analysis.

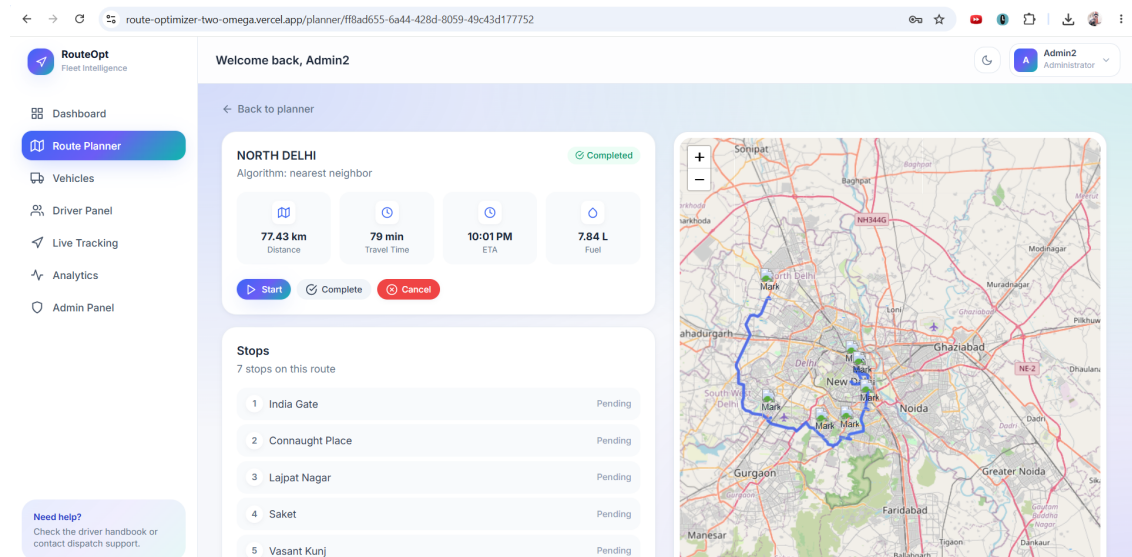


Figure 9.12: Route Planner with an interactive Leaflet map displaying the optimized delivery route, delivery stops, travel distance, estimated travel time, and fuel consumption.

CHAPTER 10

Results

This chapter discusses the overall results of the project, drawing on the testing outcomes described in Chapter 9, and reflects honestly on the system’s performance, deployment behavior, scalability characteristics, and reliability.

10.1 Performance

For the concurrency levels most representative of the system’s intended use case – a small to mid-sized delivery operation with a handful of dispatchers creating routes throughout the day, rather than hundreds of simultaneous users – the system performed comfortably, with route creation response times dominated by the OSRM round trip rather than by the application’s own logic. This confirms that isolating the routing engine call behind a dedicated service module, and keeping the optimization algorithms themselves lightweight, was a reasonable design choice: the algorithmic component of route creation is not the performance bottleneck under realistic load.

The performance and stress testing described in Sections 9.3 and 9.5 also surfaced a clear, expected limitation: at high concurrency, the single OSRM container used in this deployment becomes the primary constraint, since OSRM’s `/table` and `/route` endpoints are computationally non-trivial and the deployment used for testing ran a single instance without horizontal scaling. This is a reasonable outcome given the project’s infrastructure budget and is discussed further as a scalability consideration below.

10.2 Deployment

The split deployment model – a static frontend on Vercel and a containerized backend (with its supporting services) on Render – worked as intended in practice. Automatic deployment on push to the main branch reduced the friction of shipping incremental changes during development, and the ability to configure environment

variables independently for each platform made it straightforward to point the frontend at different backend URLs (local, staging, production) without code changes.

One practical observation from deployment was that free-tier hosting platforms, including the ones used for this project, commonly introduce cold-start latency after a period of inactivity – the first request to a backend that has been idle can take noticeably longer than subsequent requests, since the underlying container needs to be started. This did not affect the correctness of the system but is worth noting for anyone evaluating response time consistency on a similarly budgeted deployment.

10.3 Scalability

The backend was designed to be stateless with respect to application logic – no in-memory session state or route-optimization state is held between requests – which means that, in principle, additional backend instances could be run behind a load balancer to handle increased request volume. The stress test results indicate, however, that simply scaling the backend horizontally would not, by itself, resolve the bottleneck observed at high concurrency, since OSRM itself was the limiting component in this deployment. A more scalable deployment would need to run multiple OSRM instances behind their own load balancer, or scale the single OSRM instance vertically (more CPU and memory), since OSRM’s routing computations are CPU-bound.

The database layer was not observed to be a bottleneck at any tested concurrency level, which is consistent with the relatively simple query patterns used throughout the system (indexed lookups by ID and unique fields such as email and plate number, with no complex joins in the hot path of route creation).

10.4 System Reliability

The Haversine fallback path, described in Chapter 7, was the single most impactful reliability decision made in this project. Route creation depends on an external service (OSRM) that can, in principle, become temporarily unreachable due to network issues, container restarts, or maintenance. Without a fallback, this would make the system’s most important feature entirely dependent on the uptime of a separate service. With the fallback in place, route creation degrades to reduced-accuracy straight-line distances rather than failing outright, which was confirmed through manual testing (stopping the OSRM container and verifying that route

creation still succeeded, with the frontend correctly indicating that a straight-line, rather than road-based, route had been generated).

More broadly, the layered backend architecture – routes, controllers, services, and dependency-free algorithms – proved to be relatively resilient to the kind of incremental changes this project went through: the OSRM integration, the vehicle capacity fields, and the switch to email-based driver assignment were each added without requiring changes to unrelated modules, which is a reasonable indicator of a maintainable system structure, though this observation is based on the experience of a single developer working on the codebase and has not been validated across a larger team.

10.5 Summary of Results

Table 10.1 summarizes the key results discussed in this chapter against the objectives set out in Chapter 1.

Table 10.1: Summary of Results Against Objectives

Objective	Outcome
Road-based route optimization	Achieved; OSRM integration functioning with automatic fallback
Role-based dashboard for all three roles	Achieved; distinct views and permissions verified
Vehicle capacity enforcement	Achieved; validated before optimization begins
Driver assignment simplification	Achieved; email-based lookup replacing raw IDs
Analytics visibility	Achieved; dashboard and charts functioning
Containerized, reproducible deployment	Achieved; Docker Compose stack and separate production hosting both verified
Acceptable performance under realistic load	Achieved at moderate concurrency; OSRM identified as the scaling bottleneck at high concurrency

CHAPTER 11

Future Enhancements

The current implementation satisfies the core objectives set out at the start of the project, but several enhancements were identified during development and testing that were considered out of scope for this phase. This chapter presents them as planned future work rather than implemented features.

11.1 AI-Based Route Prediction

The nearest-neighbor and priority-first strategies implemented in this system are heuristic and deterministic. A natural next step would be to train a model on historical route data (once enough has been collected in production) to predict which stop sequencing strategy tends to produce better real-world outcomes for a given combination of stop count, geographic spread, and time of day, and to use that prediction to choose between – or blend – the existing strategies automatically, rather than requiring the dispatcher to select one manually.

11.2 Machine Learning for Traffic-Aware ETA

The current `trafficFactor` is a manually configured multiplier rather than a data-driven estimate. A future enhancement would replace or augment this with a machine learning model trained on historical travel time data (comparing OSRM's predicted duration against actual delivery completion timestamps recorded by the system itself) to produce time-of-day and day-of-week aware ETA adjustments, without requiring integration with a paid live-traffic API.

11.3 Electric Vehicle Route Optimization

As delivery fleets increasingly adopt electric vehicles, route optimization needs to account for constraints that do not apply to fuel-powered vehicles, such as bat-

tery range and the location of charging infrastructure. A future version of the system could extend the vehicle model with battery capacity and consumption-per-kilometre fields, and extend the optimization service to factor in range constraints and, where necessary, insert a charging stop into the route.

11.4 Demand Prediction

Currently, delivery stops are supplied for each route individually, either manually or via CSV upload. A future enhancement would analyze historical delivery patterns to predict demand for a given day or region in advance, which could help dispatchers pre-position vehicles or anticipate the number of vehicles required before the day's deliveries are even finalized.

11.5 Carbon Footprint Tracking

Since the system already estimates fuel consumption per route, extending this to estimate and aggregate carbon dioxide emissions (based on standard emissions factors per litre of fuel, and eventually per kilowatt-hour for electric vehicles) would be a relatively small addition on top of the existing fuel estimation module, and would give operators visibility into the environmental impact of their delivery operations alongside the existing cost-oriented metrics.

11.6 Additional Enhancements

Beyond the items above, the following smaller enhancements were also noted during development and testing as worthwhile future work:

- Horizontally scaling the OSRM service, as discussed in Chapter 10, to remove the bottleneck observed at high concurrency.
- Introducing a shared request-validation library (such as Zod) on the backend to formalize the input validation currently implemented as manual checks within each service function.
- Supporting multiple drivers or shift-based assignment per vehicle, rather than the current one-to-one vehicle-driver relationship.

-
- Extending vehicle capacity enforcement to also validate volume (`maxVolume`), which is currently stored but not enforced due to the absence of per-stop volume data.
 - Adding automated end-to-end tests (for example, using Playwright) to complement the manual integration testing described in Chapter 9.

CHAPTER 12

Conclusion

This project set out to build a complete, deployable route optimization system for last-mile delivery that did not depend on a commercial mapping API, and to do so with the same layered engineering discipline expected of a production system: a clearly separated backend architecture, a typed and tested optimization core, a responsive role-aware frontend, and a reproducible deployment process.

The resulting system meets that goal. It manages vehicles, drivers, and delivery routes through a role-based API; optimizes delivery sequences using either a nearest-neighbor or priority-based strategy; computes real road distances, travel times, and route geometry through a self-hosted OSRM instance, while remaining functional through an automatic fallback when that service is unavailable; enforces vehicle capacity limits before a route can be created; and gives dispatchers, drivers, and administrators each a dashboard suited to their role, including live vehicle tracking and fleet-wide analytics.

Building this system also surfaced a number of practical engineering lessons that were not obvious at the outset. The most significant of these was the importance of designing the OSRM integration with a fallback path from the very beginning, rather than treating the routing engine as an always-available dependency – this single decision materially improved the system’s reliability characteristics, as discussed in Chapter 10. Similarly, keeping the optimization algorithms free of framework and database dependencies made them trivial to unit test and easy to reason about independently of the rest of the system, which paid off directly during the incremental upgrades made throughout the project, such as switching the optimizer’s distance source from Haversine to real road distance without needing to rewrite the algorithms themselves.

The performance and security testing carried out, while not exhaustive, gave a reasonably clear picture of the system’s current limits – most notably, that a single OSRM instance becomes the practical bottleneck under high concurrent load well before the Express backend or the database do – and that picture, together with the future enhancements outlined in Chapter 11, provides a concrete basis for continuing to develop this system beyond its current state.

Overall, the project demonstrates that a genuinely useful, road-accurate route optimization system can be built entirely on open-source infrastructure, without the recurring cost of a commercial routing API, for the scale of delivery operation this system targets.

References

- [1] Project OSRM, "Open Source Routing Machine," [Online]. Available: <http://project-osrm.org/>. [Accessed: 2026].
- [2] OpenStreetMap Contributors, "OpenStreetMap," [Online]. Available: <https://www.openstreetmap.org/>. [Accessed: 2026].
- [3] Prisma Labs Inc., "Prisma ORM Documentation," [Online]. Available: <https://www.prisma.io/docs>. [Accessed: 2026].
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation," [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: 2026].
- [5] Meta Platforms Inc., "React Documentation," [Online]. Available: <https://react.dev/>. [Accessed: 2026].
- [6] Vercel Inc., "Vite Documentation," [Online]. Available: <https://vitejs.dev/>. [Accessed: 2026].
- [7] Tailwind Labs Inc., "Tailwind CSS Documentation," [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 2026].
- [8] OpenJS Foundation, "Express.js Documentation," [Online]. Available: <https://expressjs.com/>. [Accessed: 2026].
- [9] Redis Ltd., "Redis Documentation," [Online]. Available: <https://redis.io/docs/>. [Accessed: 2026].
- [10] Socket.IO, "Socket.IO Documentation," [Online]. Available: <https://socket.io/docs/>. [Accessed: 2026].
- [11] Grafana Labs, "k6 Load Testing Documentation," [Online]. Available: <https://k6.io/docs/>. [Accessed: 2026].
- [12] G. Clarke and J. W. Wright, "Scheduling of Vehicles from a Central Depot to a Number of Delivery Points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964.
- [13] S. Lin and B. W. Kernighan, "An Effective Heuristic Algorithm for the Traveling-Salesman Problem," *Operations Research*, vol. 21, no. 2, pp. 498–516, 1973.

- [14] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [15] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [16] C. G. Gonzalez, "Last-Mile Logistics: Challenges and Innovations in Urban Delivery Systems," *International Journal of Logistics Management*, vol. 32, no. 3, pp. 45–62, 2021.
- [17] Render Services Inc., "Render Documentation," [Online]. Available: <https://render.com/docs>. [Accessed: 2026].
- [18] Vercel Inc., "Vercel Documentation," [Online]. Available: <https://vercel.com/docs>. [Accessed: 2026].
- [19] Docker Inc., "Docker Compose Documentation," [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: 2026].
- [20] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: 2026].